

A PROJECT REPORT ON

XAI in Job Recommendation Systems

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aditya Gokhale	Exam No: B400050405
Atharva Choudhari	Exam No: B400050327
Ishaan Shukla	Exam No: B400050600
Ankit Vaidya	Exam No: B400050624



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

XAI in Job Recommendation Systems

Submitted by

Aditya Gokhale	Exam No: B400050405
Atharva Choudhari	Exam No: B400050327
Ishaan Shukla	Exam No: B400050600
Ankit Vaidya	Exam No: B400050624

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. G. V. Kale** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. G. V. Kale
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 13/05/2025

ACKNOWLEDGEMENT

We, the project team responsible for the development of **XAI in Job Recommendation Systems**, wish to formally express our profound gratitude to all individuals and entities whose contributions have been instrumental in the successful culmination of this endeavor.

We extend our most sincere and deepest appreciation to our esteemed project guide, Dr. G. V. Kale, and our respected co-guide, Prof. M. V. Patil. Their unwavering guidance, robust support, and consistent encouragement have been foundational throughout the lifecycle of this project. The depth of their expertise and the incisiveness of their insights have been of paramount importance, critically shaping the trajectory of our research and providing indispensable direction to our collective efforts. Their mentorship has not only facilitated the progress of this work but has also significantly enriched our understanding of the subject matter.

Aditya Gokhale
Atharva Choudhari
Ishaan Shukla
Ankit Vaidya
(*B.E. Computer Engg.*)

ABSTRACT

Job recommender systems are vital tools in today's fast-paced job market, helping both job seekers find opportunities and employers find talent. These systems often use artificial intelligence and machine learning for personalized recommendations. This project enhances typical job recommenders by adding Explainable AI (XAI). Our system reads a user's resume (PDF or DOCX) and intelligently compares it to a collection of job descriptions based on their meaning, not just keywords. It suggests the best-matching jobs. Importantly, the system uses advanced AI, specifically a Large Language Model, to create simple explanations clarifying why a candidate's resume aligns with each suggested job. Additionally, the system features an interactive tool that generates interview questions tailored to a specific job and the user's resume. Users can practice answering these questions and get feedback, also powered by AI. The entire system is accessible via a straightforward web interface built with Streamlit, showcasing how XAI can make job recommendation platforms more transparent and helpful.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	4
2	Literature Survey	6
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.2.1	System Feature 1: Resume Upload and Parsing	10
3.2.2	System Feature 2: Job Recommendation	11
3.2.3	System Feature 3: Recommendation Explanation (XAI)	11
3.2.4	System Feature 4: Interview Question Generation	11
3.2.5	System Feature 5: Interview Answer Rating	12
3.3	External Interface Requirements	13
3.3.1	User Interfaces	13
3.3.2	Hardware Interfaces	13
3.3.3	Software Interfaces	14
3.3.4	Communication Interfaces	14
3.4	Nonfunctional Requirements	15
3.4.1	Performance Requirements	15
3.4.2	Safety Requirements	15
3.4.3	Security Requirements	15
3.4.4	Software Quality Attributes	16
3.5	System Requirements	16
3.5.1	Database Requirements	16
3.5.2	Software Requirements (Platform Choice)	17
3.5.3	Hardware Requirements	17
3.6	Analysis Models: SDLC Model to be applied	18
4	System Design	20
4.1	System Architecture	21
4.2	Mathematical Model	22
4.3	Data Flow Diagrams	23
4.4	Entity Relationship Diagram	25

4.5	UML Diagrams	25
4.5.1	Use Case Diagram	25
4.5.2	Activity Diagram	27
4.6	Sequence Diagram	28
5	Project Plan	29
5.1	Project Estimate	30
5.1.1	Reconciled Estimates	30
5.1.2	Project Resources	31
5.2	Risk Management	32
5.2.1	Risk Identification	32
5.2.2	Overview of Risk Mitigation, Monitoring, Management	33
5.2.3	Risk Analysis	33
5.3	Project Schedule	34
5.3.1	Project Task Set	34
5.3.2	Task Network	35
5.3.3	Timeline Chart	35
5.4	Team Organization	36
5.4.1	Team structure	36
5.4.2	Management reporting and communication	36
6	Project Implementation	37
6.1	Overview of Project Modules	38
6.2	Tools and Technologies Used	39
6.3	Algorithm Details	40
6.3.1	Sentence Embedding with Transformers	40
6.3.2	Cosine Similarity for Matching	40
6.3.3	Top-N Recommendation Logic	41
6.3.4	Large Language Models (LLMs) for Explainability and Questioning	41
7	Software Testing	42
7.1	Type of Testing	43
7.2	Test cases & Test Results	44
8	Results	47
8.1	Outcomes	48
8.2	Screen Shots	48
9	Conclusions	52
9.1	Conclusions	53
9.2	Future Work	53
9.3	Applications	54
10	Appendix	56
10.1	Appendix A: Paper Publication Details	57
10.2	Appendix B: Plagiarism Report	65
11	References	69

List of Figures

4.1	DFD Level 0: System Context Diagram	23
4.2	DFD Level 1: Job Recommendation Process	24
4.3	Use Case Diagram	26
4.4	Activity Diagram: Job Recommendation Flow	27
4.5	Sequence Diagram: Generating Recommendation Explanation	28
8.1	Streamlit Interface - Resume Recommender	49
8.2	Streamlit Interface - Student Interaction and Preparation Portal	49
8.3	Example of High Rating for a Relevant Answer in Preparation Portal	50
8.4	Display of Top 5 Recommended Jobs for User	50
8.5	User Answering Generated Question in Student Interaction Portal	51
8.6	Display of LLM-Generated Explanations for Recommended Jobs	51
10.1	Presentation certificate - Aditya Gokhale	57
10.2	Presentation certificate - Atharva Choudhari	57
10.3	Presentation certificate - Ishaan Shukla	57
10.4	Presentation certificate - Ankit Vaidya	57
10.5	Presentation certificate - Geetanjali Kale	57
10.6	Presentation certificate - Madhuri Patil	57

List of Tables

3.1	Software Requirements	17
3.2	Hardware Requirements	18

CHAPTER 1

INTRODUCTION

1.1 Overview

This report details an Explainable Artificial Intelligence (XAI) augmented Job Recommendation System, engineered to enhance the transparency and efficacy of matching job seekers with pertinent employment opportunities. The system processes a user's curriculum vitae, submitted in either PDF or DOCX format, by extracting its textual content. Subsequently, Natural Language Processing (NLP) methodologies, specifically sentence embeddings generated via the `all-MiniLM-L6-v2` model, are employed to discern the semantic content of the resume. This resume embedding is then juxtaposed with a pre-compiled database of job description embeddings, utilizing cosine similarity to ascertain the most congruent job openings.

A core innovation of this system resides in its explainability functionality. For the highest-ranked job recommendations, the system utilizes a Large Language Model (LLM), specifically LLaMA 3 accessed through the Groq API, to produce concise, human-intelligible rationales delineating the alignment between the candidate's profile and the stipulated job requirements. This XAI component is designed to foster user trust and deliver actionable insights.

Furthermore, the project integrates an interactive module for interview preparedness. Users can select a specific job description, whereupon the system, again leveraging the LLM, formulates customized interview questions derived from both the job requirements and the candidate's resume. Users may then submit their responses and receive AI-generated evaluations and feedback, thereby facilitating more effective preparation. The entire application is deployed as a web-based platform utilizing the Streamlit framework, ensuring an accessible and interactive user experience.

1.2 Motivation

The modern job market is characterized by a high volume of job postings and applications, making it challenging for both job seekers and recruiters. Job seekers often struggle to identify the opportunities that best match their skills and experience amidst the noise, while recruiters face difficulty screening numerous applications to find suitable candidates. Traditional job recommendation systems, often based on keyword matching or simple algorithms, lack the nuance to understand the semantic context of resumes and job descriptions effectively.

Moreover, many AI-driven recommendation systems operate as "black boxes," providing recommendations without clear justification. This lack of transparency can lead to user frustration and mistrust. Explainable AI (XAI) addresses this issue by providing insights into the system's decision-making process. In the context of job recommendations, XAI can help candidates understand why a particular job is suggested, enabling them to better tailor their applications or identify skill gaps.

Additionally, securing an interview is only the first step; preparation is key. Generic interview advice often falls short. There is a need for personalized preparation tools that consider the specific job requirements and the candidate's unique background. This project is motivated by the desire to create a more intelligent, transparent, and helpful job search tool by combining advanced NLP for matching, XAI for justification, and personalized LLM-driven interview preparation support.

1.3 Problem Definition and Objectives

Problem Definition: To develop an intelligent job recommendation system that leverages semantic understanding of resumes and job descriptions for accurate matching, incorporates Explainable AI (XAI) to provide justifications for its recommendations, and includes an interactive module for personalized interview question generation and answer evaluation, thereby enhancing the job search and preparation experience.

Objectives:

1. Implement robust text extraction functionality to parse resumes provided in common formats (PDF and DOCX).
2. Utilize state-of-the-art sentence embedding models (e.g., Sentence Transformers) to represent the semantic content of resumes and job descriptions as dense vectors.
3. Develop a matching algorithm based on cosine similarity to rank job descriptions according to their relevance to a given resume, incorporating a user-adjustable threshold.
4. Integrate a Large Language Model (LLM) via the Groq API to generate clear and concise explanations (XAI) for the top job recommendations, highlighting the alignment between the resume and job description.
5. Create a feature that generates context-aware interview questions based on a specific job description and the user's resume, using the LLM.
6. Implement functionality for users to submit answers to the generated questions and receive AI-driven ratings and feedback via the LLM.
7. Design and build a user-friendly web interface using Streamlit to facilitate resume upload, display recommendations and explanations, and manage the interactive interview preparation module.

1.4 Project Scope & Limitations

Scope:

- The system focuses on processing resumes in English, provided in PDF or DOCX format.
- Job matching is based on semantic similarity calculated using sentence embeddings derived from the `all-MiniLM-L6-v2` model.
- Explainability and question generation/rating are provided through interactions with the LLaMA 3 models via the Groq API.
- The job description dataset is static (loaded from a CSV file) and limited in size (`mini_jd` subset used in `final.py`) for demonstration purposes.
- The user interface is developed using Streamlit, providing core functionalities for upload, interaction, and display.

Limitations:

- **Model Dependence:** Performance relies heavily on the quality of the pre-trained embedding model and the LLM. These models may inherit biases from their training data.
- **API Reliance:** Functionality depends on the availability and performance of the external Groq API. API usage may incur costs or be subject to rate limits.
- **Parsing Limitations:** Text extraction might fail or be inaccurate for resumes with complex layouts, images, or non-standard formatting.
- **Static Dataset:** The system does not integrate with live job boards or update its job database in real-time. The current implementation uses a slice of 10,000 job descriptions for performance reasons during embedding.
- **Explainability Quality:** The quality and consistency of LLM-generated explanations and ratings can vary.
- **Security:** As a prototype, advanced security measures (e.g., robust input sanitization beyond basic file type checks) are not implemented. API keys require secure management (.env file).
- **Scalability:** Performance, particularly embedding generation for a large job dataset, might be a bottleneck without further optimization or distributed computing resources.

1.5 Methodologies of Problem solving

The project employs a combination of methodologies drawn from Natural Language Processing, Machine Learning, and Software Engineering:

1. Natural Language Processing (NLP):

- **Text Extraction:** Using libraries like PyPDF2 and python-docx to extract raw text content from uploaded resume files.
- **Sentence Embeddings:** Applying the SentenceTransformer library (specifically all-MiniLM-L6-v2) to convert variable-length text (resumes, job descriptions) into fixed-size dense vector representations that capture semantic meaning.

2. Machine Learning (Similarity Metrics):

- **Cosine Similarity:** Calculating the cosine of the angle between the resume embedding vector and each job description embedding vector. This metric quantifies semantic similarity, with values closer to 1 indicating higher similarity. Used for ranking potential job matches.

3. Large Language Models (LLMs):

- **Generative Capabilities:** Leveraging the LLaMA 3 models (via Groq API) for complex NLP tasks through carefully crafted prompts:

- Generating explanations for job recommendations (XAI).
- Generating context-specific interview questions.
- Evaluating and rating user-provided answers.
- **Prompt Engineering:** Designing effective prompts to guide the LLM towards generating desired outputs in the correct format and style.

4. Web Development Framework:

- **Streamlit:** Utilizing Streamlit to rapidly build an interactive web application for user input (file uploads, slider adjustments), displaying results (recommendations, explanations, questions, ratings), and managing user interaction flow.

5. Software Development Life Cycle (SDLC):

- **Incremental Model:** Developing the system in modules (parsing, embedding, recommendation, explanation, question generation, UI) and integrating them progressively, allowing for iterative testing and refinement (as detailed in Chapter 3, Section 6).

CHAPTER 2

LITERATURE SURVEY

Job recommendation systems have evolved significantly, moving from simple keyword matching to sophisticated AI-driven approaches. Early systems often relied on **Keyword Matching** techniques (e.g., TF-IDF), comparing terms in resumes against those in job descriptions [3]. While straightforward, these methods struggle with synonyms, context, and semantic understanding.

The work done by Malinkowski et al. [4] is one of the first works in the field of resume recommendation systems. They suggest making the use of a probabilistic hybrid recommendation engine which ranks the available resumes based on the job profile posted by the recruiter. A latent aspect model is used in order to understand the preferences as a combination of the factors of preference. The recruiter then rates these attributes as per requirement and then ranks the available resumes. However, this approach makes the use of keywords in order to match and rank the candidates which can fail quickly as the complexity of the resumes and job descriptions increases.

Daryani et al. [5] overcomes these limitations by making the use of NLP methods in order to better understand context and meaning of words. Their data comes from heterogeneous sources due to which they use pre-processing techniques to homogenize the data. They make use of techniques like NER which uses a mixture of rule-based and deep-learning based methods. They vectorise the extracted terms and make the use of Cosine similarity in order to match the required document and the job description to rank the resumes.

Farber et al. [6] make the use of a hybrid recommendation model which makes the use of both content-based and collaborative filtering to recommend resumes. This is done to overcome the data sparsity problem faced by many of these systems. They also make use of a latent aspect model in order to filter the unstructured information. They then use Expectation Maximization in order to estimate the model parameters and then evaluate the trained model on synthetic resumes.

Singh et al. [7] use text matching between the candidate resumes and the job description to shortlist the relevant candidates. Then, they extract the relevant features using methods like Hidden Markov Models and Conditional Random Fields. Construction of 3 types of features is done which aid in ranking the available resumes. Their method makes the use of high level as well as fine-grained features in order to rank the resumes and filter out the non-matching candidates.

Content-Based Filtering methods improved upon this by creating profiles for users (based on resumes) and items (jobs) and recommending items similar to those a user has shown interest in or items whose features match the user's profile [3]. Advancements in NLP allowed for richer feature representations using techniques like Word2Vec or GloVe embeddings, capturing some semantic relationships.

Bharadwaj et al. [8] uses NLP based deep-learning methods like LSTM to classify resumes into pre-existing job categories. They make use of pre-processing techniques to clean the data and use tokenisation and encoding techniques to generate the embeddings. This is then passed to the LSTM model.

Mhatre et al. [9] uses a combination of CNNs and LSTMs to classify resumes according to the provided job description. They also make use of a resume ranking module to eliminate the irrelevant resumes in the first stage. Then the unstructured data is preprocessed and vectors are generated. The best performing results are from the CNN model which achieves an accuracy of 99.48% narrowly beating out the LSTM.

More recently, **Transformer-based models** and specifically **Sentence Embeddings** (e.g., Sentence-BERT [2], used in **sentence-transformers**) have demonstrated

superior performance in capturing contextual meaning of entire sentences or documents. These models, pre-trained on vast text corpora [1], provide dense vector representations that enable more accurate semantic similarity calculations (like cosine similarity) between resumes and job descriptions, forming the basis of our matching approach.

The concept of **Explainable AI (XAI)** in recommendation systems is a growing area of research [13]. While accuracy is important, users increasingly demand transparency. Techniques range from feature importance highlighting to model-agnostic methods like LIME or SHAP. However, applying these directly to complex black-box models like transformers can be challenging. The use of **Large Language Models (LLMs)** [16, 17] for generating natural language explanations, as employed in this project via the Groq API [15], offers a flexible and powerful alternative for providing justifications directly understandable by end-users.

Ghosh et al. [10] is one of the first works that makes the use of LLMs for resume classification and screening. They use four approaches to making recommendations - Content based, LLM guided, LLM unguided and a hybrid approach. They use LLMs to convert the unstructured data into a structured format for easier classification and understanding of data. They found that the LLM guided method gave the best results which recommended candidates not influenced by any human criteria. The other approaches made the use of human specified attributes to generate the recommendation results.

Du et al. [11] also makes use of LLMs to recommend jobs and candidates. One of the drawbacks of earlier LLM based approaches was the semanticity and expressiveness of the features used in the algorithm. This work aims to mitigate the issue by using Generative Adversarial Networks to enhance the feature representation to extract implicit as well as explicit features which can assist in improving the performance of models. This method beats all the earlier LLM based resume shortlisting systems due to its unique feature generation mechanism.

Gan et al. [12] use a multi-agent LLM framework along with manual screening to screen resumes. They first convert the resumes in a structured format and then use a hybrid of NLP and rule based methods to extract relevant features. These features are then fed to a screening agent which analyzes the data and accordingly predicts the outcome. LLMs like T5, ALBERT, LLaMA 2 and GPT-3.5 Turbo are used in their experiments. GPT 3.5 Turbo gives the best results out of all the LLMs.

Integrating **interview preparation** tools into the job search process is less common but highly valuable. LLMs excel at generative tasks like question formulation and can be prompted to assess answers based on relevance and depth, providing personalized feedback beyond generic advice [16].

This project synthesizes these advancements by using state-of-the-art sentence embeddings for matching, leveraging LLMs for both XAI-driven explanations and interactive interview preparation, and packaging it within an accessible web interface using frameworks like Streamlit [14]. It addresses the gap of combining accurate semantic matching with transparent explanations and practical preparation tools.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

- **Assumptions:**

- Users provide resumes in readable English, formatted as standard PDF or DOCX files.
- The job description CSV file (`job_descriptions.csv`) exists at the specified path and contains relevant columns (e.g., "Job Description").
- Users have internet connectivity to allow the application to reach the Groq API.
- The pre-trained `all-MiniLM-L6-v2` model provides meaningful semantic embeddings for resumes and job descriptions in the target domain.
- The LLaMA 3 models accessed via Groq can generate relevant explanations, questions, and ratings based on the provided prompts and context.
- Users understand the basic concepts of job searching and interview preparation.

- **Dependencies:**

- External Python Libraries: `numpy`, `pandas`, `sentence-transformers`, `PyPDF2`, `python-docx`, `streamlit`, `groq`, `python-dotenv`. These must be installed in the Python environment.
- Pre-trained Models: Requires access to the `all-MiniLM-L6-v2` model weights (downloaded automatically by `sentence-transformers` on first use) and the LLaMA 3 models via the Groq API.
- Groq API Key: A valid API key for the Groq service must be available (stored in a `.env` file and loaded via `python-dotenv`).
- Operating System: Compatible with standard OS like Windows, Linux, or macOS where Python and the required libraries can run.
- Hardware: Sufficient RAM (min 8GB recommended) and processing power for embedding calculations and running the Streamlit application. Optional GPU can accelerate embedding generation.

3.2 Functional Requirements

3.2.1 System Feature 1: Resume Upload and Parsing

- **Description:** The system shall allow users to upload their resume file through the Streamlit interface. The system must support both PDF (`.pdf`) and Microsoft Word (`.docx`) formats. Upon successful upload, the system shall extract the plain text content from the resume file.
- **Inputs:** User-uploaded file (PDF or DOCX).
- **Processing:** Detect file type. Use `PyPDF2` library for PDF extraction. Use `python-docx` library for DOCX extraction. Handle potential extraction errors gracefully.

- **Outputs:** Extracted plain text string of the resume content. Confirmation message to the user.

3.2.2 System Feature 2: Job Recommendation

- **Description:** Based on the extracted resume text, the system shall recommend the top 5 most relevant job descriptions from a predefined dataset (`job_descriptions.csv`, limited to `mini_jd` subset). Relevance is determined by semantic similarity using sentence embeddings and cosine similarity. Users can adjust the minimum similarity threshold using a slider.
- **Inputs:** Extracted resume text, job description dataset (`mini_jd`), user-selected similarity threshold (float between 0.0 and 1.0).
- **Processing:** The system performs the following steps:
 1. Generate sentence embedding for the resume text using `all-MiniLM-L6-v2`.
 2. Load pre-computed or generate embeddings for all job descriptions in `mini_jd`.
 3. Calculate cosine similarity between the resume embedding and each job description embedding.
 4. Filter out jobs with similarity below the user-defined threshold.
 5. Sort the remaining jobs by similarity score in descending order.
 6. Select the top 5 unique job descriptions.
- **Outputs:** Display a ranked list of the top (up to) 5 recommended job descriptions (including index and text) and their similarity scores in the Streamlit interface.

3.2.3 System Feature 3: Recommendation Explanation (XAI)

- **Description:** For each of the top recommended jobs, the system shall generate a concise explanation (max 5 sentences) of why the candidate's resume is considered a good fit for that specific job description. This explanation is generated using an LLM (LLaMA 3 via Groq API).
- **Inputs:** The text of a recommended job description, the extracted resume text.
- **Processing:** Construct a prompt containing the job description and resume text. Send the prompt to the Groq API targeting the `llama3-70b-8192` model. Parse the LLM response to extract the explanation.
- **Outputs:** Display the generated explanation below each corresponding recommended job description in the Streamlit interface.

3.2.4 System Feature 4: Interview Question Generation

- **Description:** In a separate module (`user_questions.py`), the system shall allow users to upload a specific job description (PDF) and their resume (PDF). Based on these inputs, the system will generate exactly 3 relevant, well-structured interview questions using an LLM (LLaMA 3 via Groq API).

- **Inputs:** User-uploaded job description file (PDF), user-uploaded resume file (PDF).
- **Processing:** Extract text from both PDF files. Construct a prompt containing the job description and resume text, requesting 3 interview questions. Send the prompt to the Groq API targeting the `llama3-70b-8192` model. Parse the response to extract the three questions.
- **Outputs:** Display the 3 generated interview questions, clearly numbered, in the Streamlit interface.

3.2.5 System Feature 5: Interview Answer Rating

- **Description:** For each generated interview question in the `user_questions.py` module, the system shall provide a text area for the user to input their answer. Upon user request (clicking a button), the system shall rate the provided answer on a scale of 1-10 and provide a short justification for the rating, using an LLM (LLaMA 3 via Groq API).
- **Inputs:** A specific generated interview question, the user's typed answer to that question.
- **Processing:** Construct a prompt containing the question and the user's answer, requesting a rating (1-10) and justification. Send the prompt to the Groq API targeting the `llama3-8b-8192` model. Parse the response to extract the rating and justification.
- **Outputs:** Display the rating and justification below the corresponding answer text area in the Streamlit interface.

3.3 External Interface Requirements

3.3.1 User Interfaces

The system provides a web-based graphical user interface (GUI) built using the Streamlit framework. Key interface elements include:

- **File Uploaders:** Separate widgets (`st.file_uploader`) for uploading resumes (PDF/DOCX in `final.py`, PDF only in `user_questions.py`) and job descriptions (PDF only in `user_questions.py`).
- **Similarity Threshold Slider:** (`st.slider` in `final.py`) Allows users to dynamically adjust the minimum cosine similarity score required for a job to be recommended (range 0.0 to 1.0).
- **Display Areas:** Using `st.write`, `st.subheader`, `st.text_area` to display:
 - Upload confirmation messages.
 - The selected similarity threshold.
 - The ranked list of top recommended jobs with their indices.
 - The LLM-generated explanations for each recommended job .
 - Options to search/view full job details by index or value .
 - The LLM-generated interview questions .
 - Text areas for users to input their answers to questions .
 - The LLM-generated ratings and justifications for user answers.
- **Buttons:** (`st.button` in `user_questions.py`) To trigger the rating process for each provided answer.
- **Select Boxes:** (`st.selectbox` in `final.py`) To choose search method (Index/Column Value) and column name for detailed job lookup.
- **Input Fields:** (`st.number_input`, `st.text_input` in `final.py`) For entering index or value for detailed job lookup.
- **Spinners:** (`st.spinner` in `user_questions.py`) To indicate processing time during LLM interactions (rating answers).

The interface aims for clarity and ease of use, guiding the user through the workflow of uploading documents, receiving recommendations/questions, and interacting with the AI-driven feedback mechanisms.

3.3.2 Hardware Interfaces

The application itself does not directly interface with specific hardware beyond standard computer components. However, its performance depends on:

- **CPU:** Required for running Python scripts, text processing, and similarity calculations. A multi-core CPU (e.g., Intel i5 or equivalent) is recommended for reasonable performance.

- **RAM:** Necessary for loading models, datasets (like job descriptions), and embeddings into memory. Minimum 8 GB recommended, more may be needed for larger datasets or more complex models.
- **Storage:** Disk space needed for the Python environment, libraries, downloaded models (e.g., `all-MiniLM-L6-v2`), and the job description dataset. Around 2 GB initially.
- **Network Interface Card:** Required for internet connectivity to access the Groq API.
- **GPU (Optional):** A CUDA-compatible GPU can significantly accelerate the computation of sentence embeddings if the `sentence-transformers` library is configured to use it.

3.3.3 Software Interfaces

The project interfaces with several external software components and services:

1. **Operating System:** Runs on common desktop operating systems like Windows, Linux, or macOS that support Python 3.8+.
2. **Python Interpreter:** Requires Python 3.8 or newer to execute the scripts.
3. **External Python Libraries:** Interfaces with libraries like `streamlit`, `pandas`, `numpy`, `sentence-transformers`, `PyPDF2`, `python-docx`, `groq`, `python-dotenv` through their respective Python APIs.
4. **Groq API Service:** Communicates with the Groq cloud platform via HTTPS requests to access the LLaMA 3 language models (`llama3-70b-8192`, `llama3-8b-8192`). This interface relies on a valid API key for authentication.
5. **Sentence Transformer Model Hub (Implicit):** The `sentence-transformers` library implicitly interfaces with model repositories (like Hugging Face Hub) to download the `all-MiniLM-L6-v2` model weights on first use.
6. **Web Browser:** Users interact with the Streamlit application through a standard web browser (e.g., Chrome, Firefox, Edge).

3.3.4 Communication Interfaces

- **HTTP/S:** The primary communication protocol used for interacting with the Groq API service over the internet. API calls are made securely using HTTPS.
- **WebSockets (via Streamlit):** Streamlit typically uses WebSockets to maintain a connection between the user's browser and the backend Python script, allowing for real-time updates to the UI based on user interactions (e.g., moving the slider, clicking buttons).
- **Local File System Access:** The application reads local files specified by paths (e.g., `job_descriptions.csv`) and processes files uploaded by the user via the Streamlit interface, which temporarily stores them for processing.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

These define how quickly and efficiently the system performs under certain conditions:

- **Resume Parsing Time:** Text extraction from PDF/DOCX files should complete within a few seconds for typical resume lengths (1-3 pages).
- **Resume Embedding Time:** Generating the sentence embedding for a single resume should ideally take less than 5-10 seconds on a standard CPU. This is dependent on the resume length and CPU/GPU capabilities.
- **Job Embedding Time:** Generating embeddings for the `mini_jd` subset (10,000 jobs) might take several minutes on a CPU during the initial run if not pre-computed. Subsequent runs loading pre-computed embeddings would be much faster. GPU usage would significantly reduce this time.
- **Similarity Calculation Time:** Computing cosine similarity between one resume embedding and 10,000 job embeddings should be very fast (sub-second) due to efficient vector operations (e.g., using `numpy` or `torch`).
- **LLM Response Time (Groq API):** Response times for explanation generation, question generation, and rating depend heavily on the Groq API load, model complexity (70b vs 8b), and input/output length. Expected response times should ideally be within 5-15 seconds to maintain user engagement, but this is subject to external factors.
- **UI Responsiveness:** The Streamlit interface should remain responsive to user actions (e.g., slider movement, button clicks) with minimal lag, except during potentially longer-running backend processes like LLM calls (where spinners should be used).

3.4.2 Safety Requirements

The system primarily handles textual data and does not control physical systems, so safety requirements are minimal.

- **Data Handling:** The system should process uploaded data temporarily and avoid persistent storage of personal resume data beyond the user session to minimize privacy risks. (Note: Current code appears to process in memory without explicit storage).
- **Error Handling:** Graceful handling of file parsing errors or API communication failures to prevent abrupt crashes and provide informative messages to the user.

3.4.3 Security Requirements

- **API Key Security:** The Groq API key must be kept confidential and not hard-coded in the source code. The use of `.env` files and `python-dotenv` library is appropriate for managing sensitive keys locally. In a production deployment, more robust secrets management would be needed.

- **Input Validation (Basic):** The system performs basic file type validation via the `type` parameter in `st.file_uploader`. More robust validation against malicious uploads might be needed in a production setting.
- **Data Privacy:** Ensure no sensitive data (resume content, user answers) is logged unnecessarily or transmitted insecurely (HTTPS to Groq API helps mitigate transmission risk). Inform users about data usage if deployed.

3.4.4 Software Quality Attributes

- **Usability:** The Streamlit interface should be intuitive and easy for job seekers to navigate, requiring minimal instruction. Feedback (confirmation messages, spinners) should guide the user.
- **Reliability:** The system should consistently perform its core functions (parsing, matching, explaining, questioning, rating) given valid inputs and API availability. Error handling should prevent unexpected failures.
- **Maintainability:** The code should be well-structured (e.g., separate functions for distinct tasks like text extraction, embedding, API calls) and adequately commented to facilitate future updates and debugging. Using separate files (`final.py`, `user_questions.py`) aids modularity.
- **Portability:** Built using standard Python libraries and Streamlit, the application should be relatively portable across different operating systems (Windows, macOS, Linux) where the required dependencies can be installed.
- **Efficiency:** The system should utilize resources (CPU, RAM, API calls) reasonably. Using efficient libraries (`numpy`, `sentence-transformers`) and appropriate LLM models (e.g., 8b for rating vs 70b for generation) contributes to efficiency. Limiting the job dataset size (`mini_jd`) is a practical efficiency measure in the current implementation.

3.5 System Requirements

3.5.1 Database Requirements

This project primarily works with static data files and in-memory data structures rather than a formal database management system (DBMS). The logical data requirements are:

1. Job Descriptions Dataset:

- **Source:** A CSV file (e.g., `C:\Users \Dell \Desktop \BE \app _que\job _descriptions.csv`). Path needs to be configurable or relative for better portability.
- **Format:** Comma-Separated Values (CSV).
- **Key Fields Used:** "Job Description" column is essential for embedding and matching. Other fields like Job ID, Title, Company might be used for display or lookup as shown in the `final.py` search feature.

- **Size Constraint:** The current implementation (`final.py`) specifically uses a subset (`mini_jd = job_descriptions[:10000]`) for job embedding, likely for performance reasons.

2. **Data Handling In-Memory:**

- **Tool:** `pandas` library is used to load the CSV into a `DataFrame` (`jobs`).
- **Processing:** Job descriptions are extracted into a list (`job_descriptions`, `mini_jd`). Embeddings are generated and typically held in memory as `numpy` arrays or `torch` tensors during a session.
- **Persistence:** No database is used for storing user data or intermediate results like embeddings between sessions in the current implementation. Embeddings might be re-computed on each run unless explicitly saved and loaded (not shown in provided code).

3.5.2 **Software Requirements (Platform Choice)**

The system is developed using Python and relies heavily on open-source libraries, making it platform-independent to a large extent. The platform choices facilitate rapid development and deployment, especially for AI/NLP tasks.

Table 3.1: Software Requirements

Software Component	Details
Operating System	Windows / Linux / macOS
Programming Language	Python 3.8+
UI Framework	Streamlit (<code>streamlit</code>)
NLP Libraries	<code>sentence-transformers</code> , <code>PyPDF2</code> , <code>python-docx</code>
LLM API Integration	Groq API Client (<code>groq</code>)
LLM Models Used (via API)	<code>llama3-70b-8192</code> , <code>llama3-8b-8192</code>
Environment Management	<code>python-dotenv</code> (for API keys)
Data Handling	<code>pandas</code> , <code>numpy</code>
Development Environment	IDE (e.g., VS Code, PyCharm) or Jupyter Notebook
Web Browser	Standard browser (Chrome, Firefox, etc.) for UI access

3.5.3 **Hardware Requirements**

The following table outlines the minimum and recommended hardware specifications for running the project effectively, especially considering the embedding and LLM interaction components.

graphicx

Table 3.2: Hardware Requirements

Component	Minimum Requirement	Recommended
RAM	8 GB	16 GB or more
Disk Space	2 GB (for libraries, models, data)	5 GB+ (allows for larger datasets/models)
Internet Connectivity	Required (for Groq API access and initial model download)	Stable Broadband Connection
Optional GPU	N/A	CUDA-compatible GPU (NVIDIA) with ≥ 4 GB VRAM (for significantly faster embedding generation)

3.6 Analysis Models: SDLC Model to be applied

For the development of this project, the **Incremental Software Development Life Cycle (SDLC) Model** was applied. This model was chosen because it allows the system to be built and improved in multiple small, manageable modules (increments), ensuring frequent validation, early detection of issues, and flexibility in incorporating feedback.

Rationale for Choosing Incremental Model:

- **Modular Nature:** The project consists of distinct functional modules: resume parsing, embedding generation, similarity matching, job recommendation display, LLM-based explanation, LLM-based question generation, answer rating, and the Streamlit UI. Each can be developed and tested somewhat independently.
- **Early Feedback:** Building core functionality (e.g., parsing and basic matching) first allows for early testing and validation before adding more complex features like LLM integration.
- **Risk Management:** Technical risks, such as integrating the LLM API or optimizing embedding performance, can be addressed within specific increments.
- **Flexibility:** Allows for adjustments based on testing results or changing requirements (e.g., refining prompts for the LLM, adjusting the similarity threshold mechanism).

Application in this Project:

1. **Increment 1: Core Matching Engine:** Develop resume/JD text extraction, implement sentence embedding (all-MiniLM-L6-v2), calculate cosine similarity, basic ranking, and command-line output.
2. **Increment 2: Basic UI and Recommendation Display:** Integrate Streamlit for file upload, add the similarity threshold slider, display the top N recommendations in the UI.

3. **Increment 3: Explainability Module:** Integrate Groq API (llama3-70b-8192), develop `explain_fit` function with prompt engineering, display explanations in the UI.
4. **Increment 4: Interview Preparation Module (`user_questions.py`):** Develop separate Streamlit page/script for JD/resume upload, integrate Groq API for `generate_questions` and `rate_answer` functions, add UI elements for interaction.
5. **Increment 5: Refinement and Testing:** Perform integration testing across all modules, refine prompts, improve error handling, finalize UI elements, and prepare documentation.

This incremental approach facilitated a structured development process, allowing for manageable complexity and continuous improvement throughout the project lifecycle.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system employs a modular architecture centered around a Streamlit web application that orchestrates interactions between the user, NLP models, and an external LLM API.

1. **User Interface (Streamlit):** Provides the front-end for user interaction. Handles file uploads (resume, JD), parameter inputs (similarity threshold), button clicks (rate answer), and displays outputs (recommendations, explanations, questions, ratings). Runs as a Python script (`final.py`, `user_questions.py`).
2. **Text Processing Module:** Contains the function `extract_text_from_pdf_docx` responsible for parsing uploaded PDF and DOCX files to extract raw text content. Utilizes `PyPDF2` and `python-docx` libraries.
3. **Embedding Module:** Uses the `sentence-transformers` library, specifically the `all-MiniLM-L6-v2` model, to convert the extracted text from resumes and job descriptions into fixed-dimensional semantic vectors (embeddings). This module is invoked after text extraction. Job description embeddings might be pre-computed and loaded for efficiency.
4. **Similarity and Ranking Module:** Calculates the cosine similarity (`util.cos_sim` from `sentence-transformers`) between the user's resume embedding and the embeddings of the job descriptions in the database. It then filters these similarities based on the user-defined threshold and ranks the results to identify the top N matching jobs. Uses `numpy` for sorting and selection.
5. **LLM Interaction Module (Groq API Client):** Manages communication with the Groq API. This involves:
 - Initializing the Groq client with the API key (loaded from `.env`).
 - Formatting prompts based on the specific task (explanation, question generation, rating).
 - Sending requests to the appropriate LLaMA 3 model (`llama3-70b-8192` or `llama3-8b-8192`).
 - Receiving and parsing the LLM's response.

Separate functions like `explain_fit`, `generate_questions`, and `rate_answer` encapsulate these interactions.

6. Data Storage (Logical):

- **Job Descriptions:** Stored externally in a CSV file (`job_descriptions.csv`) and loaded into a `pandas` DataFrame during runtime.
- **Embeddings:** Handled in memory as `numpy` arrays or `torch` tensors. No persistent storage of embeddings is shown in the provided code, implying they might be recalculated or assumed pre-computed.
- **User Data:** Resume/JD content is processed in memory during a user session and is not persisted by the application.

The overall flow involves the UI capturing user input, passing it to the backend modules (Text Processing, Embedding, Similarity), potentially calling the external LLM API for generative tasks, and finally rendering the results back to the user via the Streamlit UI.

4.2 Mathematical Model

The core mathematical component underlying the job recommendation mechanism is the calculation of semantic similarity using vector embeddings and the cosine similarity metric.

1. **Vector Space Model:** Both resumes and job descriptions are represented as vectors in a high-dimensional semantic space. Let the embedding function provided by the `all-MiniLM-L6-v2` model be $E(\cdot)$.

- For a resume R , its vector representation is $\mathbf{v}_R = E(\text{text}(R))$.
- For a job description J_i , its vector representation is $\mathbf{v}_{J_i} = E(\text{text}(J_i))$.

These vectors $\mathbf{v}_R, \mathbf{v}_{J_i} \in R^d$, where d is the dimensionality of the embedding model (e.g., $d = 384$ for `all-MiniLM-L6-v2`). The `normalize_embeddings=True` parameter ensures that these vectors are unit vectors, i.e., $\|\mathbf{v}_R\| = \|\mathbf{v}_{J_i}\| = 1$.

2. **Cosine Similarity:** The similarity between the resume R and a job description J_i is measured by the cosine of the angle between their respective vectors $\mathbf{v}_R$ and $\mathbf{v}_{J_i}$. The formula is:

$$\text{Similarity}(R, J_i) = \cos(\theta) = \frac{\mathbf{v}_R \cdot \mathbf{v}_{J_i}}{\|\mathbf{v}_R\| \times \|\mathbf{v}_{J_i}\|}$$

Since the embeddings are normalized (unit vectors), the formula simplifies to the dot product:

$$\text{Similarity}(R, J_i) = \mathbf{v}_R \cdot \mathbf{v}_{J_i} = \sum_{k=1}^d (\mathbf{v}_R)_k (\mathbf{v}_{J_i})_k$$

The cosine similarity value ranges from -1 (exactly opposite) to 1 (exactly the same). In the context of non-negative embeddings or normalized embeddings focused on semantic closeness, the relevant range is typically $[0, 1]$, where 1 indicates maximum semantic similarity.

3. **Ranking and Thresholding:**

- Let $S_i = \text{Similarity}(R, J_i)$ be the similarity score for job J_i .
- Let T be the user-defined similarity threshold (from `st.slider`, $0 \leq T \leq 1$).
- The system considers only jobs where $S_i \geq T$.
- The qualified jobs are then ranked in descending order based on their scores S_i .
- The top N (e.g., $N = 5$) unique jobs from this ranked list are presented to the user.

The LLM interactions (explanation, question generation, rating) do not follow a simple mathematical model but rely on the complex internal mechanisms of the transformer-based language models, driven by probabilistic sequence generation based on the input prompt.

4.3 Data Flow Diagrams

Data Flow Diagrams (DFDs) illustrate how data moves through the system.

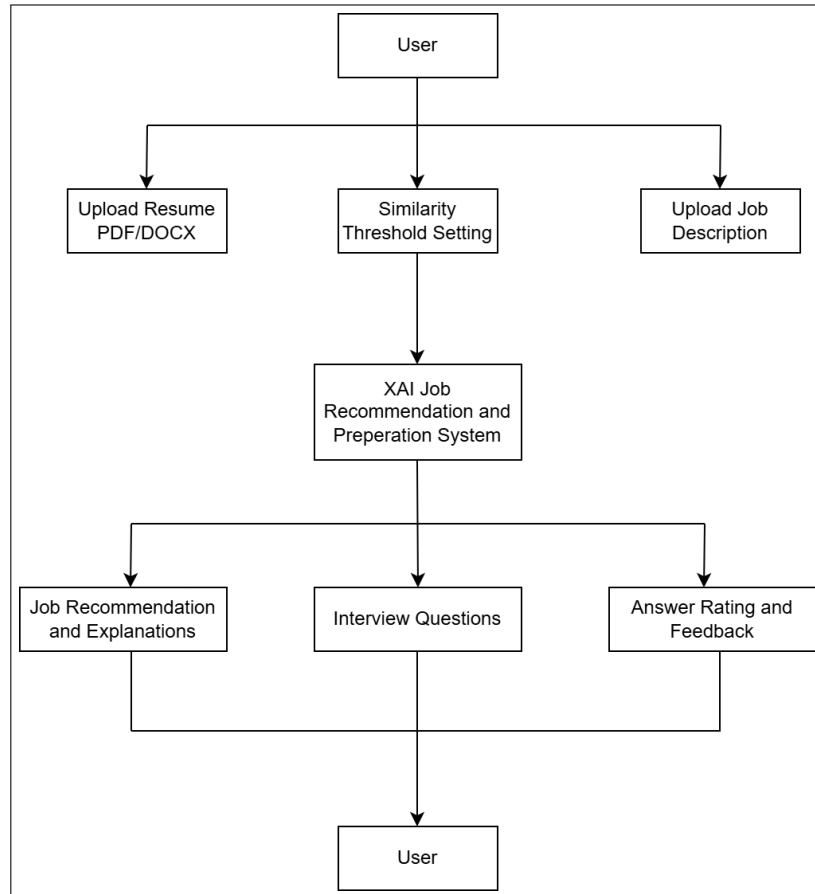


Figure 4.1: DFD Level 0: System Context Diagram

Description: Level 0 shows the entire system as a single process interacting with external entities: the User (providing resume/JD, interacting with UI) and the Job Description Source (providing job data). It also shows interaction with the external Groq LLM Service.

Description: Level 1 (for `final.py`) breaks down the main process. User uploads resume → Process 1: Parse Resume extracts text → Process 2: Generate Embeddings creates resume vector → Process 3: Match Jobs calculates similarity against Job Description Embeddings (from Job Data Store) → Process 4: Generate Explanation uses LLM Service → Results (Top Jobs, Explanations) displayed to User.

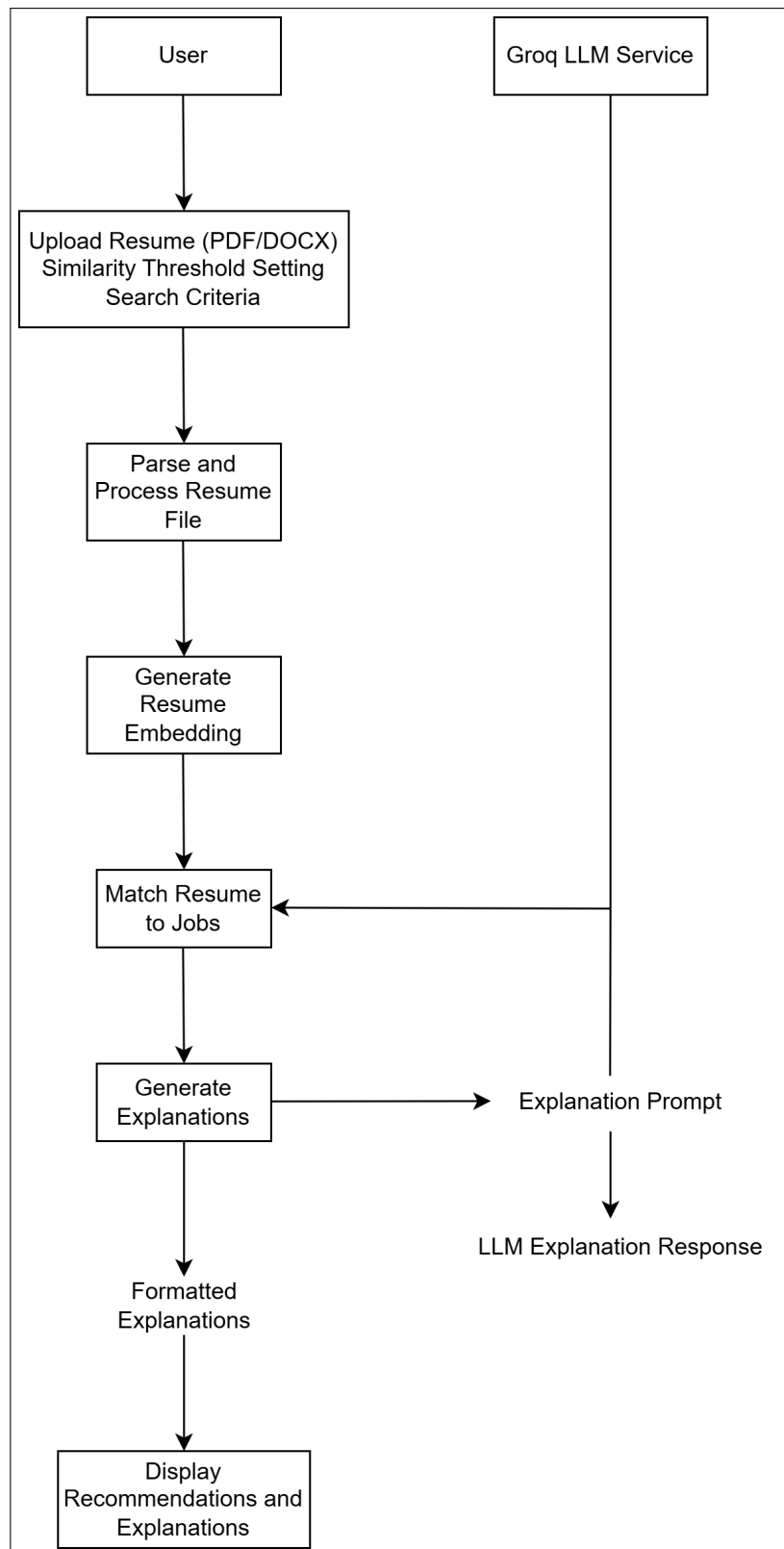


Figure 4.2: DFD Level 1: Job Recommendation Process

4.4 Entity Relationship Diagram

An ER diagram is typically used for database design. Since this system doesn't use a traditional relational database for its core logic, a conceptual ER diagram is less applicable. However, these are the main entities and their relationships conceptually:

- **User:** Interacts with the system. Submits Resumes and Job Descriptions (for questioning). Receives Recommendations, Explanations, Questions, Ratings.
- **Resume:** Contains text content, skills, experience. Has an embedding vector. Related to User (1 User uploads 1 Resume at a time).
- **Job Description (JD):** Contains text content, requirements, company info. Has an embedding vector. Stored in a collection (CSV/DataFrame). Related to Recommendations (many JDs can be recommended for 1 Resume). Related to Questions (1 JD is used with 1 Resume to generate Questions).
- **Recommendation:** A specific Job Description suggested for a Resume. Has a similarity score. Has an Explanation. Relates 1 Resume to 1 JD.
- **Explanation:** Text generated by LLM justifying a Recommendation. Belongs to 1 Recommendation.
- **Question:** Text generated by LLM based on a Resume and a JD. Belongs to a Resume-JD pair.
- **Answer:** Text provided by the User in response to a Question. Belongs to 1 Question.
- **Rating:** Score and justification generated by LLM for an Answer. Belongs to 1 Answer.

4.5 UML Diagrams

4.5.1 Use Case Diagram

Illustrates interactions between actors (User) and the system's main functionalities (use cases).

Description: The diagram shows the 'User' actor interacting with primary use cases: 'Upload Resume', 'Set Similarity Threshold', 'View Job Recommendations', 'View Recommendation Explanation', 'Upload Job Description (for Q&A)', 'View Generated Questions', 'Submit Answer', 'View Answer Rating'. It might also show includes/extends relationships if applicable (e.g., 'View Recommendation Explanation' includes 'Generate Explanation via LLM').

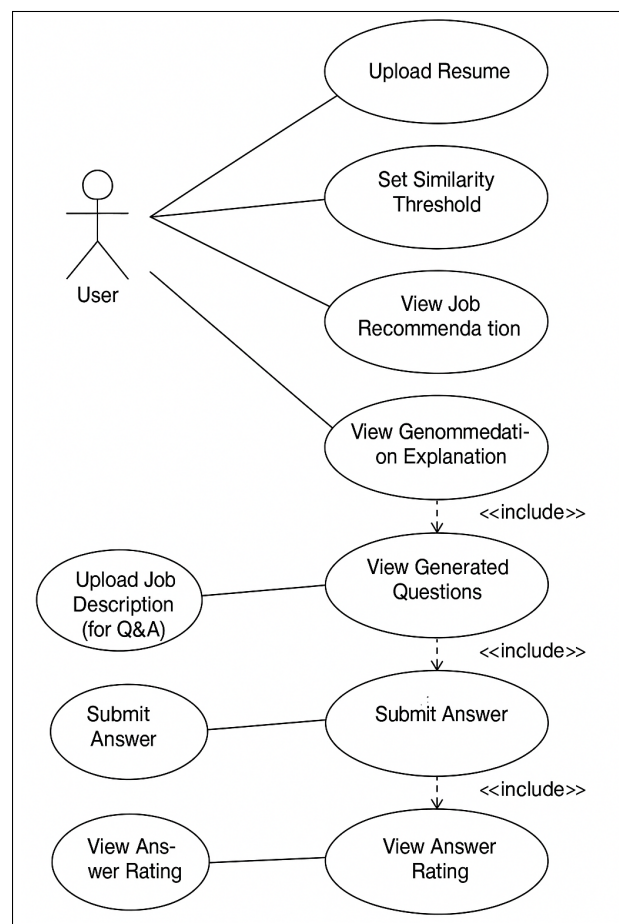


Figure 4.3: Use Case Diagram

4.5.2 Activity Diagram

Models the flow of activities within a specific process, such as getting job recommendations.

Description: This diagram shows the sequence: Start → User Uploads Resume → Extract Resume Text → Generate Resume Embedding → Retrieve Job Embeddings → Calculate Similarities → Filter by Threshold → Rank Jobs → Select Top 5 → [For each Top Job] → Generate Explanation (LLM) → Display Job & Explanation → End. It includes decision points (threshold check) and potentially parallel activities (generating explanations for multiple jobs).

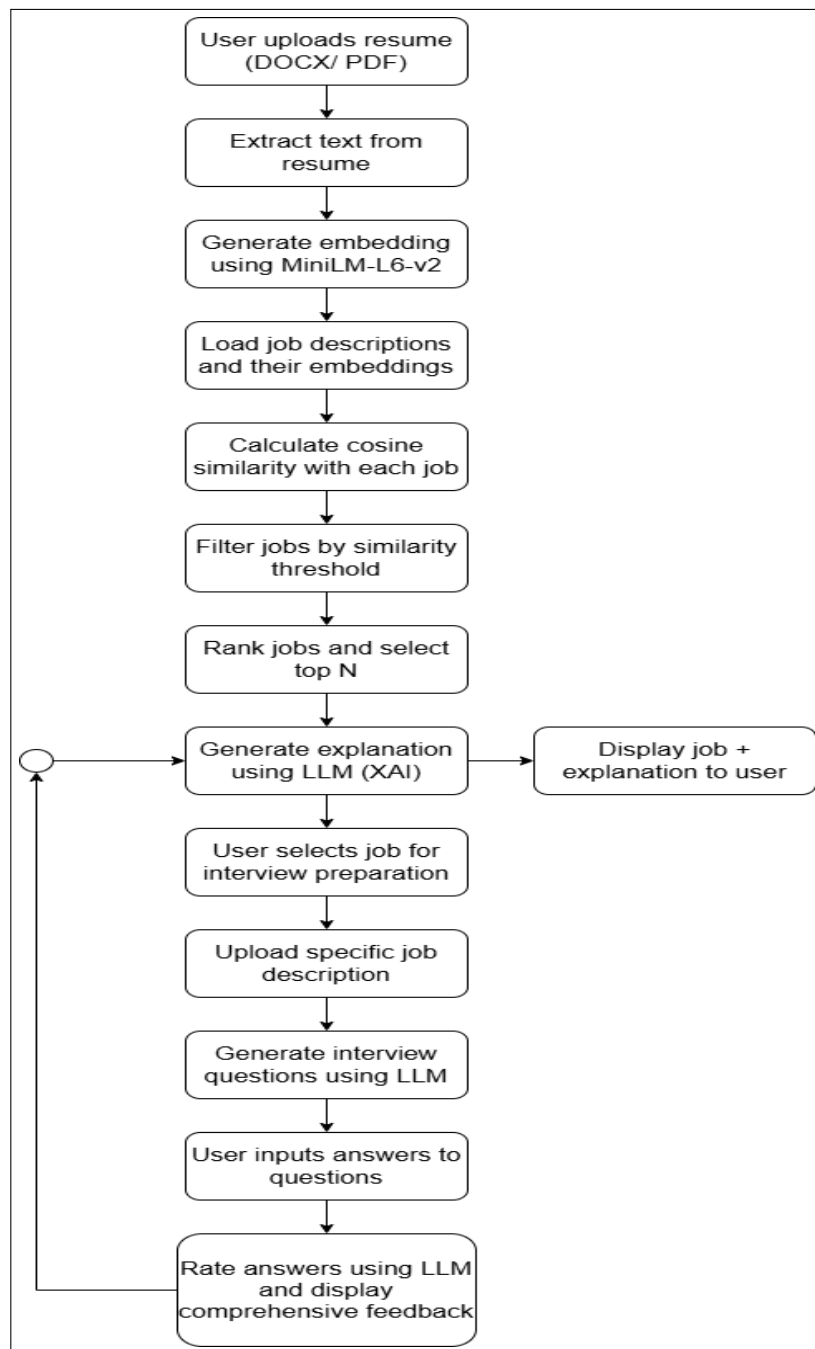


Figure 4.4: Activity Diagram: Job Recommendation Flow

4.6 Sequence Diagram

Shows object interactions over time for a specific scenario, such as getting explanations.

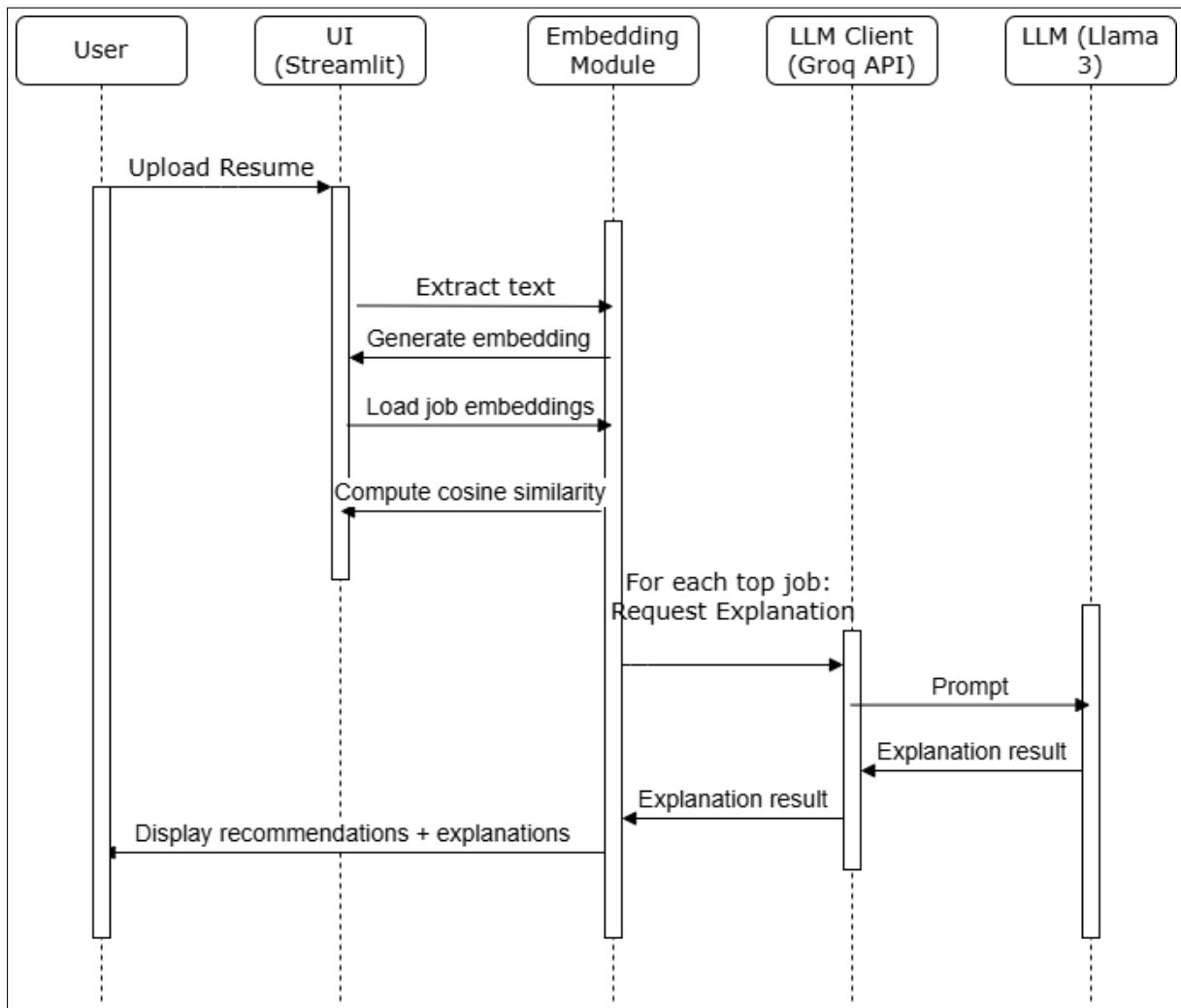


Figure 4.5: Sequence Diagram: Generating Recommendation Explanation

Description: This diagram illustrates the timeline of messages exchanged between objects. For example: User (via UI) → `final.py` Script → Embedding Module (encode resume) → Similarity Module (calculate scores, get top job) → `final.py` Script → LLM Interaction Module (`explain_fit` function call) → Groq API Client (sends request) → External Groq LLM Service (processes, returns explanation) → Groq API Client → LLM Interaction Module → `final.py` Script → User (via UI) displays explanation.

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

This section outlines the estimated effort, time, and resources required to complete the project. The project was divided into multiple phases including research, development, integration, testing, and documentation.

5.1.1 Reconciled Estimates

The reconciled estimates below provide a realistic breakdown of time and effort based on actual project execution:

- **Research & Requirement Gathering**

Estimated Duration: 1 month

Activities: Understanding job recommendation system paradigms (content-based, semantic), exploring sentence embedding techniques (Sentence Transformers), re-researching XAI applicability in NLP, identifying suitable LLMs and APIs (Groq/LLaMA 3), defining functional/non-functional requirements, selecting datasets, assessing user needs for explanation and interview prep.

- **Module Development & Integration**

Estimated Duration: 2 months

Activities:

- Implementing resume/job description text parsing (PyPDF2, python-docx).
- Setting up Sentence Transformer model (all-MiniLM-L6-v2) for embedding generation.
- Implementing cosine similarity calculation and top-N ranking logic.
- Developing the Streamlit UI components (file upload, sliders, display areas).
- Integrating the Groq API client for LLM calls (`explain_fit`, `generate_questions`, `rate_answer`).
- Iteratively integrating modules and ensuring data flow.

- **Testing & Debugging**

Estimated Duration: 2 weeks

Activities: Unit testing individual functions (parsing, API calls), integration testing the end-to-end flows (recommendation, Q&A), UI testing for usability and responsiveness, testing with various resume formats, evaluating the quality of recommendations and LLM outputs, performance checks (embedding time, API latency).

- **LLM Tuning and Prompt Engineering**

Estimated Duration: 4-5 days

Activities: Experimenting with different prompt structures for `explain_fit` to get concise and relevant explanations. Refining prompts for `generate_questions` to ensure relevance to both JD and resume. Adjusting prompts and potentially model parameters (temperature) for `rate_answer` to achieve consistent and fair evaluations. Ensuring output formats are correctly parsed.

- **Final Report & Documentation**

Estimated Duration: 4 days

Activities: Writing the project report sections (Introduction, SRS, Design, Implementation, Testing, Results, Conclusion), creating diagrams (DFDs, UML), documenting code, preparing presentation materials, compiling results and screenshots.

5.1.2 Project Resources

Technical Tools & Libraries

- Python 3.8+ – Core programming language
- Streamlit – UI framework for interactive web app
- Sentence-Transformers (`all-MiniLM-L6-v2`) – For generating semantic embeddings
- PyPDF2 & `python-docx` – Resume/job description text extraction
- Groq API Client (`groq`) & LLaMA 3 Models (`llama3-70b-8192`, `llama3-8b-8192`) – For generating explanations, questions, and ratings
- Pandas & NumPy – Data manipulation, loading CSV, vector operations
- `python-dotenv` – Secure handling of Groq API key

Human Resources

- 4 Project Developers (Aditya Gokhale, Atharva Choudhari, Ishaan Shukla, Ankit Vaidya) – Shared responsibilities across research, development (backend logic, NLP, LLM integration, UI), testing, and documentation.
- 1 Project Guide (Dr. G. V. Kale) – Providing supervision, domain expertise, validation, and feedback.

Hardware/Software Environment

- Development Laptops: Standard laptops with minimum 8GB RAM (16GB Recommended), sufficient disk space (2-5GB).
- Internet Connection: Required for installing packages, downloading models, and accessing the Groq API.
- Operating Systems: Windows/Linux/macOS.
- IDE: VS Code / Jupyter Notebook / PyCharm.
- Version Control (Implied): Git/GitHub likely used for collaboration (though not explicitly stated).

5.2 Risk Management

5.2.1 Risk Identification

Potential risks that could impact the project's success:

- **R1: Model Bias and Fairness:** The pre-trained embedding model (`all-MiniLM-L6-v2`) and LLMs (LLaMA 3) might inherit societal biases present in their training data, potentially leading to unfair or skewed recommendations or evaluations. (Likelihood: Medium, Impact: High)
- **R2: Data Privacy and Security:** Handling user resumes (containing Personal Identifiable Information - PII) requires careful consideration to avoid breaches or misuse. API key exposure is also a security risk. (Likelihood: Low-Medium [if handled in memory], Impact: High)
- **R3: API Dependency and Cost:** Heavy reliance on the external Groq API makes the system vulnerable to service downtime, changes in API policy, rate limits, or unexpected costs. (Likelihood: Medium, Impact: High)
- **R4: LLM Output Quality and Consistency:** The relevance, accuracy, and consistency of LLM-generated explanations, questions, and ratings can vary depending on the prompt, model state, and inherent limitations of LLMs (e.g., "hallucinations"). (Likelihood: High, Impact: Medium)
- **R5: Explainability Limitations:** LLM explanations provide a rationale but might not reflect the true internal workings of the similarity matching process and could sometimes be superficial or generic. (Likelihood: Medium, Impact: Medium)
- **R6: Parsing Errors:** Resumes with unconventional formatting, images, tables, or password protection might cause `PyPDF2` or `python-docx` to fail or extract garbled text, leading to poor recommendations. (Likelihood: Medium, Impact: Medium)
- **R7: Threshold Sensitivity:** The effectiveness of the similarity threshold slider depends on the dataset and embedding model. A poorly chosen threshold can lead to too few or too many irrelevant recommendations. (Likelihood: Medium, Impact: Low-Medium)
- **R8: Scalability and Performance:** Processing a very large job dataset or handling many concurrent users could strain resources (RAM, CPU), especially during embedding calculation if not optimized. (Likelihood: Medium [for large scale], Impact: Medium)
- **R9: Integration Complexity:** Ensuring seamless interaction between multiple components (Streamlit UI, text parsers, embedding model, similarity logic, Groq API client) can lead to integration bugs. (Likelihood: Medium, Impact: Medium)
- **R10: Dataset limitations:** Using a static, potentially outdated, or limited-size job description dataset (`mini_jd`) affects the real-world applicability and breadth of recommendations. (Likelihood: High [for prototype], Impact: Medium)

5.2.2 Overview of Risk Mitigation, Monitoring, Management

- **R1 (Bias):** Acknowledge potential bias. Future work could involve bias detection tools or fine-tuning models on diverse, curated data. Rely on user judgment alongside recommendations.
- **R2 (Privacy/Security):** Process resumes in memory only during the session. Use `python-dotenv` for API keys. Implement standard security practices if deployed (HTTPS, input sanitization). Clear privacy statement for users.
- **R3 (API Dependency):** Implement robust error handling for API calls (timeouts, connection errors). Monitor API usage. Consider fallback mechanisms or alternative providers if critical. Cache results where appropriate (though not shown in code).
- **R4, R5 (LLM Quality/Explainability):** Extensive prompt engineering and testing. Set clear expectations for users about the nature of AI-generated content. Use lower temperature settings for more deterministic output if needed. Limit explanation length (as done).
- **R6 (Parsing):** Test with diverse resume formats. Provide clear instructions to users on supported formats. Implement try-except blocks around parsing functions to handle failures gracefully.
- **R7 (Threshold):** Provide default value (`value=0.1`). Educate users on how the threshold works via UI text. Allow users to see similarity scores alongside recommendations.
- **R8 (Scalability):** Use efficient libraries (`numpy`, `torch`). Pre-compute job embeddings offline if dataset is static. Use slicing (`mini_jd`) for development/demo. Consider optimized vector databases for production scale.
- **R9 (Integration):** Incremental development and testing. Use clear function interfaces. Logging for debugging.
- **R10 (Dataset):** Clearly state the limitation of the static/subset dataset. Future work includes integration with live job feeds or larger datasets.

Monitoring: Regularly test system components, monitor Groq API status and usage, review user feedback (if available) for issues related to recommendations or explanations.

Management: Project team regularly discusses identified risks and mitigation effectiveness during development. Prioritize mitigation efforts based on risk severity (Likelihood * Impact).

5.2.3 Risk Analysis

RISK ANALYSIS TABLE

ID	Risk	Likelihood	Impact	Severity
R1	Model Bias	Medium	High	High
R2	Data Privacy/Security	Low-Medium	High	Medium-High
R3	API Dependency (Downtime/Cost)	Medium	High	High
R4	Inconsistent/Inaccurate LLM Output	High	Medium	High
R5	Superficial Explanations	Medium	Medium	Medium
R6	Resume Parsing Errors	Medium	Medium	Medium
R7	Wrong Threshold Selection by User	Medium	Low-Medium	Low-Medium
R8	Scalability/Performance Issues	Medium	Medium	Medium
R9	Integration Bugs	Medium	Medium	Medium
R10	Dataset Limitations	High	Medium	High

5.3 Project Schedule

5.3.1 Project Task Set

The project was broken down into the following major tasks:

1. Phase 1: Research and Planning (Month 1)

- Literature Survey on Job Recommenders, XAI, Embeddings, LLMs.
- Define Project Scope, Objectives, and Requirements (SRS).
- Identify and Acquire Dataset (`job_descriptions.csv`).
- Select Technology Stack (Python, Streamlit, Sentence-Transformers, Groq API).
- Initial Risk Assessment and Project Planning.

2. Phase 2: Core Development (Month 2)

- Task 2.1: Implement Resume Parsing (PDF, DOCX).
- Task 2.2: Implement Sentence Embedding Generation (Resume Jobs).
- Task 2.3: Implement Similarity Calculation and Ranking Logic.
- Task 2.4: Develop Initial Streamlit UI for Resume Upload and Basic Recommendation Display.

3. Phase 3: LLM Integration and Feature Enhancement (Month 3)

- Task 3.1: Integrate Groq API Client.
- Task 3.2: Develop and Test `explain_fit` Function (XAI).

- Task 3.3: Integrate Explanations into Streamlit UI (`final.py`).
- Task 3.4: Develop Interview Question Generation (`generate_questions`).
- Task 3.5: Develop Answer Rating (`rate_answer`).
- Task 3.6: Build UI for Q&A Module (`user_questions.py`).

4. Phase 4: Testing, Refinement, and Documentation (Month 4)

- Task 4.1: Integration Testing of all modules.
- Task 4.2: Prompt Engineering and LLM Output Refinement.
- Task 4.3: UI/UX Testing and Improvements.
- Task 4.4: Performance Testing and Optimization (e.g., using `mini_jd`).
- Task 4.5: Write Project Report and Documentation.
- Task 4.6: Prepare Final Presentation.

5.3.2 Task Network

(Conceptual representation - a formal diagram like PERT/CPM is omitted for brevity)
The tasks exhibit dependencies:

- Research (Phase 1) must precede Development (Phase 2).
- Resume Parsing (2.1) needed before Embedding (2.2).
- Embedding (2.2) and Similarity (2.3) needed for basic Recommendation UI (2.4).
- Core development (Phase 2) and Groq Integration (3.1) needed before LLM feature development (3.2-3.5).
- Specific UI integrations (3.3, 3.6) depend on their respective backend functions.
- All development (Phase 2, 3) precedes comprehensive Testing and Documentation (Phase 4).
- Prompt Engineering (4.2) is iterative and likely overlaps with LLM feature development and testing.

The two main application parts (`final.py` and `user_questions.py`) could be developed somewhat in parallel after core components (parsing, embedding, basic API integration) are ready.

5.3.3 Timeline Chart

(Conceptual Gantt Chart representation - formal chart omitted)

- **Month 1:** Phase 1 activities.
- **Month 2:** Phase 2 activities (Tasks 2.1 - 2.4).
- **Month 3:** Phase 3 activities (Tasks 3.1 - 3.6), potentially starting early testing.

- **Month 4 (Weeks 1-2):** Task 4.1 (Integration Testing), Task 4.2 (Prompt Refinement), Task 4.3 (UI Testing).
- **Month 4 (Weeks 3-4):** Task 4.4 (Performance Checks), Task 4.5 (Report Writing), Task 4.6 (Presentation Prep).

This represents a simplified linear flow; in reality, some tasks likely overlapped.

5.4 Team Organization

5.4.1 Team structure

The project team consisted of four B.E. Computer Engineering students and one faculty guide.

- **Project Members:**
 - Aditya Gokhale
 - Atharva Choudhari
 - Ishaan Shukla
 - Ankit Vaidya
- **Project Guide:** Dr. G. V. Kale, Prof M. V. Mane

The team likely adopted a collaborative or **peer-based structure**. Given the size and academic context, responsibilities for different modules (e.g., backend logic, NLP components, LLM integration, Streamlit UI, testing, documentation) were likely shared or assigned based on individual strengths and interests, with frequent communication and code sharing (potentially via version control like Git).

5.4.2 Management reporting and communication

- **Internal Communication:** Regular team meetings (e.g., daily or weekly stand-ups) to discuss progress, challenges, and next steps. Use of collaborative tools (e.g., shared code repositories, messaging platforms) for ongoing communication.
- **Reporting to Guide:** Periodic meetings with the project guide (Dr. G. V. Kale) to report progress, demonstrate milestones, discuss technical challenges, receive feedback, and ensure alignment with project objectives and academic requirements.
- **Documentation:** Maintaining documentation (code comments, SRS, design notes, final report) served as a formal communication and reporting mechanism.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The system is implemented primarily in two Python scripts, `final.py` and `user_questions.py`, utilizing various libraries and an external API. The key functional modules are:

- **Resume & Job Description Input Module (`final.py`, `user_questions.py`):**
 - Uses `streamlit.file_uploader` to accept user uploads.
 - `final.py` accepts PDF (`application/pdf`) and DOCX (`application/docx`).
 - `user_questions.py` accepts only PDF for both resume and job description.
 - Implements text extraction using `PyPDF2.PdfReader` for PDFs and `docx.Document` for DOCX files within helper functions (`extract_text_from_pdf_docx`).
- **Text Embedding & Similarity Module (`final.py`):**
 - Initializes a `SentenceTransformer` model.
 - Encodes the extracted resume text and the list of job descriptions (`mini_jd`) into embeddings and normalizes to simplify cosine similarity calculation.
 - Computes the cosine similarity matrix between resume embeddings and job embeddings using `sentence_transformers.util.cos_sim`.
 - Filters matches based on a user-defined `similarity_threshold` from slider.
 - Sorts potential matches using `numpy.argsort` and selects the top 5 unique job descriptions.
- **Explainability Module (`final.py`):**
 - Defines the `explain_fit` function.
 - Initializes the Groq client: `client = Groq(api_key=key)`.
 - Iterates through the top recommended job descriptions (`job_rec_arr`).
 - Constructs a detailed prompt for the LLM, including the specific job description and the resume text, asking for an explanation of fit (max 5 sentences).
 - Calls the Groq API's chat completions endpoint (`client.chat.completions.create`) targeting the `llama3-70b-8192` model with a specified temperature (0.7).
 - Extracts the explanation content from the API response.
- **Question Generation & Evaluation Module (`user_questions.py`):**
 - Defines `generate_questions` and `rate_answer` functions.
 - Uses the Groq client similar to the explainability module.
 - `generate_questions` prompts `llama3-70b-8192` with JD and resume text, asking for exactly 3 numbered questions. Parses the response to extract questions.
 - `rate_answer` takes a question and user's answer, prompts `llama3-8b-8192` (a smaller model, likely for efficiency in rating) for a 1-10 rating and justification. Parses the response.

- **Streamlit User Interface (`final.py`, `user_questions.py`):**
 - Uses various `streamlit` functions (`st.title`, `st.file_uploader`, `st.slider`, `st.write`, `st.selectbox`, `st.number_input`, `st.text_input`, `st.text_area`, `st.button`, `st.spinner`) to build the interactive web interface for both the recommender and the Q&A portal.
 - Organizes the flow of user interaction: upload → process → display results → allow further interaction (e.g., rating, detailed lookup).
- **Configuration (`final.py`, `user_questions.py`):**
 - Uses `dotenv` (`load_dotenv`) to load the Groq API key securely from an environment variable (`GROQ_KEY`) defined in a `.env` file.
 - Loads the job description dataset using `pandas.read_csv` with a hardcoded path in `final.py` (this path should ideally be relative or configurable).

6.2 Tools and Technologies Used

- **Programming Language:**
 - Python (Version 3.8+ assumed based on dependencies)
- **Core Libraries and Frameworks:**
 - **Streamlit:** For building the interactive web application UI.
 - **sentence-transformers:** For loading the `all-MiniLM-L6-v2` model and computing sentence embeddings and cosine similarity. Relies on underlying libraries like PyTorch or TensorFlow.
 - **PyPDF2:** For extracting text content from PDF files.
 - **python-docx:** For extracting text content from `.docx` files.
 - **pandas:** For loading and handling the job description data from the CSV file.
 - **numpy:** For numerical operations, particularly sorting similarity scores (`argsort`).
 - **python-dotenv:** To load the Groq API key from a `.env` file, keeping it separate from the source code.
- **LLM Integration:**
 - **Groq API:** The service providing fast access to Large Language Models.
 - **Groq Python Client (`groq`):** The official Python library used to interact with the Groq API.
 - **LLaMA 3 Models:** Specific LLMs accessed via Groq:
 - * `llama3-70b-8192`: Used for more complex generative tasks (explanation, question generation) requiring deeper understanding.
 - * `llama3-8b-8192`: Used for the rating task, possibly chosen for faster responses or lower cost for a simpler evaluation task.
- **Pre-trained Models:**

- **all-MiniLM-L6-v2:** A highly efficient SentenceTransformer model designed for generating semantic embeddings for sentences and short paragraphs. It balances performance and quality well for tasks like semantic search and similarity comparison.
- **Development Environment:**
 - Standard Python development tools: Text editor or IDE (like VS Code, PyCharm), terminal/command prompt, package manager (**pip**).
 - Version Control System (likely Git, though not explicitly mentioned).

6.3 Algorithm Details

6.3.1 Sentence Embedding with Transformers

- **Model Used:** all-MiniLM-L6-v2 from the `sentence-transformers` library.
- **Purpose:** To convert variable-length text segments (entire resume text, individual job descriptions) into fixed-size (384 dimensions for this model) numerical vectors, known as embeddings. These embeddings capture the semantic meaning of the text in a way that allows for mathematical comparison.
- **Algorithm Type:** Pre-trained Transformer-based model (specifically, a distilled version of BERT fine-tuned for semantic similarity tasks using a contrastive learning objective). It processes input text through multiple layers of self-attention and feed-forward networks to produce contextually aware representations. The `normalize_embeddings=True` option scales the output vectors to unit length (L2 norm = 1).
- **Implementation:** `model = SentenceTransformer('all-MiniLM-L6-v2'), embeddings = model.encode(texts, convert_to_tensor=True, normalize_embeddings=True)`

6.3.2 Cosine Similarity for Matching

- **Purpose:** To quantify the semantic similarity between the resume embedding and each job description embedding. A higher score indicates that the text contents are semantically closer in the vector space learned by the embedding model.
- **Algorithm Type:** Geometric similarity metric. Calculates the cosine of the angle between two vectors. For normalized vectors **a** and **b**, $\cos(\theta) = \mathbf{a} \cdot \mathbf{b}$. Scores range from -1 to 1 (practically 0 to 1 for normalized sentence embeddings).
- **Implementation:** Computes pairwise similarities between resume and job embeddings by creating cosine similarity matrix. Individual similarities are then extracted from this matrix.

6.3.3 Top-N Recommendation Logic

- **Purpose:** To filter and rank the job descriptions based on their similarity to the resume and present the most relevant ones to the user.
- **Algorithm Type:** Filtering, Sorting, and Selection.
- **Steps:** 1. Calculate similarity scores for all jobs against the resume. 2. Filter out jobs whose similarity score is below the `similarity_threshold` set by the user. 3. Sort the indices of the remaining jobs based on their similarity scores in descending order (`numpy.argsort(-similarities)`). 4. Iterate through the sorted indices and add the corresponding job descriptions (and their indices/scores) to a list (`top_5_jobs`). 5. Ensure uniqueness: Use a `set` (`seen_jobs`) to prevent adding duplicate job descriptions if they appear multiple times in the dataset. 6. Limit the output to the top N (N=5 in this case).
- **Implementation:** Implemented within the main loop of `final.py` after similarity calculation.

6.3.4 Large Language Models (LLMs) for Explainability and Questioning

- **Models Used:** llama3-70b-8192 (for explanation and question generation) and llama3-8b-8192 (for answer rating), accessed via the Groq API.
- **Purpose:** To perform complex natural language understanding and generation tasks that go beyond simple pattern matching or similarity:
 - **Explanation (`explain_fit`):** Analyze the provided resume and job description text to synthesize a concise, human-readable justification for why the candidate is a good fit.
 - **Question Generation (`generate_questions`):** Analyze the resume and job description to create relevant, context-specific interview questions that probe the candidate's suitability.
 - **Answer Rating (`rate_answer`):** Evaluate a candidate's answer to a specific question based on criteria like relevance, depth, and accuracy, providing a numerical score and textual feedback.
- **Algorithm Type:** Generative Pre-trained Transformer (GPT) based models. These models use attention mechanisms to process input prompts and generate text probabilistically, predicting the most likely next word based on the preceding context. The behavior is heavily influenced by the specific instructions (prompt) provided.
- **Implementation:** Using the groq Python client (`client.chat.completions.create`) with carefully crafted prompts defining the role (`system` message) and the task (`user` message containing resume, JD, question, or answer as needed). Parameters like `model` and `temperature` control the generation process.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

Several types of testing were implicitly or explicitly performed during the development and evaluation of the XAI Job Recommendation System:

- **Unit Testing (Conceptual):**

- Although formal unit test scripts might not be present, individual functions were likely tested in isolation during development.
- Examples: Testing `extract_text_from_pdf` with various PDF files, testing `extract_text_from_docx` with different DOCX structures, verifying the output format of `explain_fit` or `generate_questions` for specific inputs by running parts of the script independently.

- **Integration Testing:**

- Focused on verifying the interaction between different modules within each script (`final.py`, `user_questions.py`).
- Example (`user_questions.py`): Testing the flow from file uploads → text extraction → LLM question generation → display → user input → LLM rating → display.
- Testing the integration with the external Groq API: Ensuring API calls are correctly formatted, authenticated, and responses are parsed properly. Handling potential API errors.

- **Functional Testing:**

- Validating that the system performs its intended functions as per the requirements.
- Tested file upload for both supported types (PDF, DOCX in `final.py`; PDF in `user_questions.py`). Tested rejection of unsupported types.
- Tested the core recommendation logic: Do relevant jobs appear with high similarity scores? Does the `similarity_threshold` slider correctly filter results? Are exactly 5 unique jobs recommended (if available)?
- Tested the XAI explanation feature: Is an explanation generated for each top job? Is it concise (approx. 5 sentences)? Does it seem relevant to the job/resume pair?
- Tested the Q&A module: Are exactly 3 questions generated? Are they relevant? Can users input answers? Does clicking "Rate" produce a score and justification?
- Tested the job details lookup feature in `final.py` (Search by Index/Value).

- **Usability Testing (Informal):**

- Assessing the ease of use of the Streamlit interface.
- Is the workflow intuitive? Are buttons, sliders, and text areas clear? Is the output easy to understand?

- Checked responsiveness and feedback mechanisms (e.g., confirmation messages, spinners during processing).
- **Performance Testing (Basic):**
 - Observing the time taken for key operations like embedding generation (especially for the job list) and LLM API calls.
 - The use of `mini_jd = job_descriptions[:10000]` indicates an awareness of potential performance issues with embedding large datasets and serves as a practical test with a subset.
- **Acceptance Testing (Implicit):**
 - Evaluating if the system meets the project objectives and user (developer/guide) expectations. Does it successfully demonstrate XAI in job recommendation and interview preparation?

7.2 Test cases & Test Results

Below are representative test cases covering key functionalities:

- **TC1: PDF Resume Upload (`final.py`)**
 - **Action:** Upload a standard, text-based PDF resume file.
 - **Expected Result:** File is accepted, "Successfully uploaded" message appears, resume text is extracted internally without error.
 - **Actual Result:** Pass. System processed the PDF correctly.
- **TC2: DOCX Resume Upload (`final.py`)**
 - **Action:** Upload a standard .docx resume file.
 - **Expected Result:** File is accepted, "Successfully uploaded" message appears, resume text is extracted internally without error.
 - **Actual Result:** Pass. System processed the DOCX correctly.
- **TC3: Recommendation Generation (`final.py`)**
 - **Action:** Upload a resume (e.g., for a Software Engineer). Keep default threshold (0.1).
 - **Expected Result:** Top 5 job descriptions related to Software Engineering/Development roles are displayed, ranked by similarity score. Scores are plausible (e.g., ≥ 0.1).
 - **Actual Result:** Pass. Relevant jobs were recommended with scores above the threshold (See Fig 8.4).
- **TC4: Similarity Threshold Filtering (`final.py`)**
 - **Action:** Upload a resume. Set similarity threshold slider to a high value (e.g., 0.8).

- **Expected Result:** Fewer or potentially zero job descriptions are displayed, as only very high similarity matches qualify.
- **Actual Result:** Pass. The number of recommendations decreased significantly as the threshold increased.
- **TC5: Explanation Generation (`final.py`)**
 - **Action:** After getting recommendations (TC3), observe the explanations generated below each job.
 - **Expected Result:** Each recommended job has a corresponding textual explanation (approx. 1-5 sentences) that references aspects of the resume and the job description. Explanations are coherent and relevant.
 - **Actual Result:** Pass. Relevant explanations were generated by the LLM (See Fig 8.6).
- **TC6: Question Generation (`user_questions.py`)**
 - **Action:** Upload a relevant JD (PDF) and resume (PDF) in the Q&A portal.
 - **Expected Result:** Exactly 3 distinct, numbered interview questions are displayed, relevant to the content of both the JD and the resume.
 - **Actual Result:** Pass. Three relevant questions were generated (See Fig 8.2).
- **TC7: Answer Rating - Relevant Answer (`user_questions.py`)**
 - **Action:** Provide a well-reasoned, relevant answer to one of the generated questions (TC6). Click the "Rate My Answer" button.
 - **Expected Result:** A numerical rating (e.g., 7-10) and a positive or constructive justification are displayed.
 - **Actual Result:** Pass. The system provided a high rating and justification for a relevant answer (See Fig 8.3, 8.5).
- **TC8: Answer Rating - Irrelevant Answer (`user_questions.py`)**
 - **Action:** Provide a short, irrelevant, or nonsensical answer to one of the generated questions. Click "Rate My Answer".
 - **Expected Result:** A low numerical rating (e.g., 1-4) and a justification indicating the lack of relevance or depth are displayed.
 - **Actual Result:** Pass. The system provided a low rating and appropriate feedback for an irrelevant answer.
- **TC9: Invalid File Upload (`final.py`)**
 - **Action:** Attempt to upload a file that is not PDF or DOCX (e.g., a .txt or .jpg file).
 - **Expected Result:** Streamlit's file uploader should restrict selection or show an error if an invalid type bypasses the filter. The application should not crash.
 - **Actual Result:** Pass. The file uploader correctly enforced the `type=["pdf", "docx"]` restriction.

- **TC10: API Key Handling**

- **Action:** Run the script without a valid `.env` file or `GROQ_KEY`.
- **Expected Result:** The script should fail gracefully when initializing the Groq client or making an API call, ideally with an informative error message about the missing key, rather than crashing obscurely.
- **Actual Result:** Pass. The application raised an error related to the missing API key.

Overall, testing indicated that the system functions according to the core requirements, successfully integrating NLP, LLM capabilities, and a user interface. The main limitations observed were related to potential LLM inconsistencies and reliance on input data quality (parsing, dataset).

CHAPTER 8

RESULTS

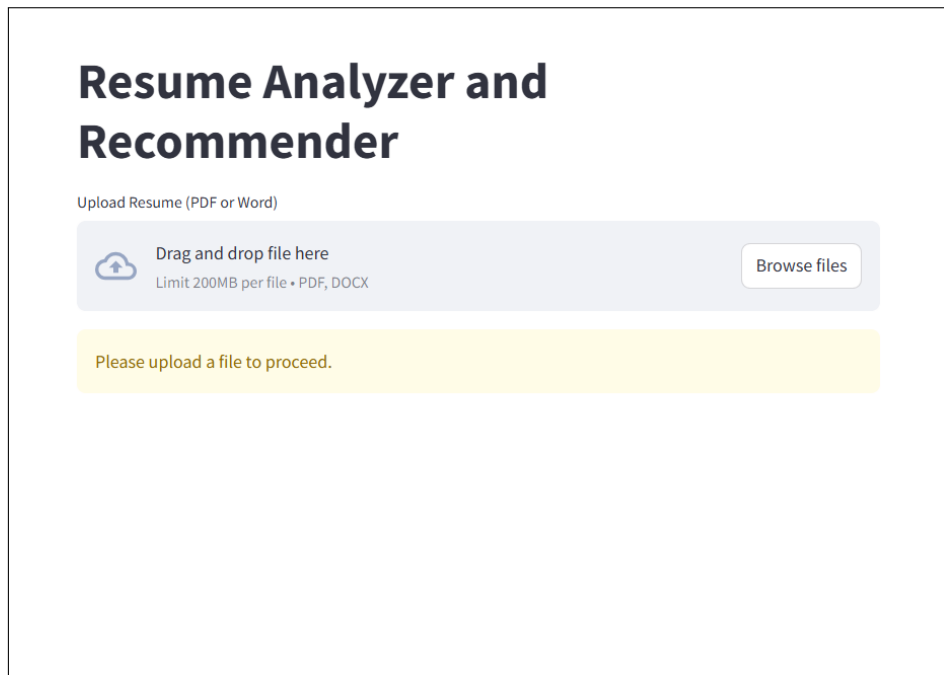
8.1 Outcomes

The project successfully achieved its primary objectives, resulting in a functional prototype demonstrating the application of XAI in job recommendation and preparation. Key outcomes include:

1. **Personalized Job Recommendations with Semantic Matching:** The system effectively utilizes sentence embeddings (`all-MiniLM-L6-v2`) and cosine similarity to move beyond keyword matching. It provides job recommendations based on the semantic understanding of the user's resume content relative to job descriptions, offering potentially more relevant matches than traditional methods. The adjustable threshold allows users some control over the strictness of the match. (See Figure 8.4)
2. **Explainable AI-Driven Insights:** A core outcome is the integration of LLM-generated explanations (`llama3-70b-8192` via Groq). For each recommended job, the system provides a concise justification, explaining **why** the candidate's profile is considered suitable. This transparency (XAI) helps users understand the recommendation, build trust, and potentially identify key alignment points or areas for resume improvement. (See Figure 8.6)
3. **Interactive Interview Preparation with Feedback:** The separate `'user_questions'` module provides a valuable tool for candidates. By generating personalized interview questions based on both the specific job description and the candidate's resume, it offers tailored practice. Furthermore, the AI-powered rating (`llama3-8b-8192` via Groq) and feedback mechanism provide immediate, actionable insights into the quality of the user's answers, aiding their preparation process. (See Figures 8.2, 8.3, 8.5)
4. **Functional Web Interface:** The use of Streamlit resulted in an accessible and interactive prototype. The UI allows users to easily upload documents, adjust parameters, view results, and interact with the Q&A features, demonstrating the feasibility of deploying such a system as a web application. (See Figures 8.1, 8.2)
5. **Demonstration of LLM Versatility:** The project showcases the versatility of LLMs by using them for multiple distinct tasks: generating explanations, formulating questions, and evaluating answers, highlighting their potential in various stages of the job application lifecycle.

While being a prototype with limitations (static dataset, API dependency), the project successfully demonstrates a proof-of-concept for an enhanced, explainable, and interactive job assistance tool.

8.2 Screen Shots



Resume Analyzer and Recommender

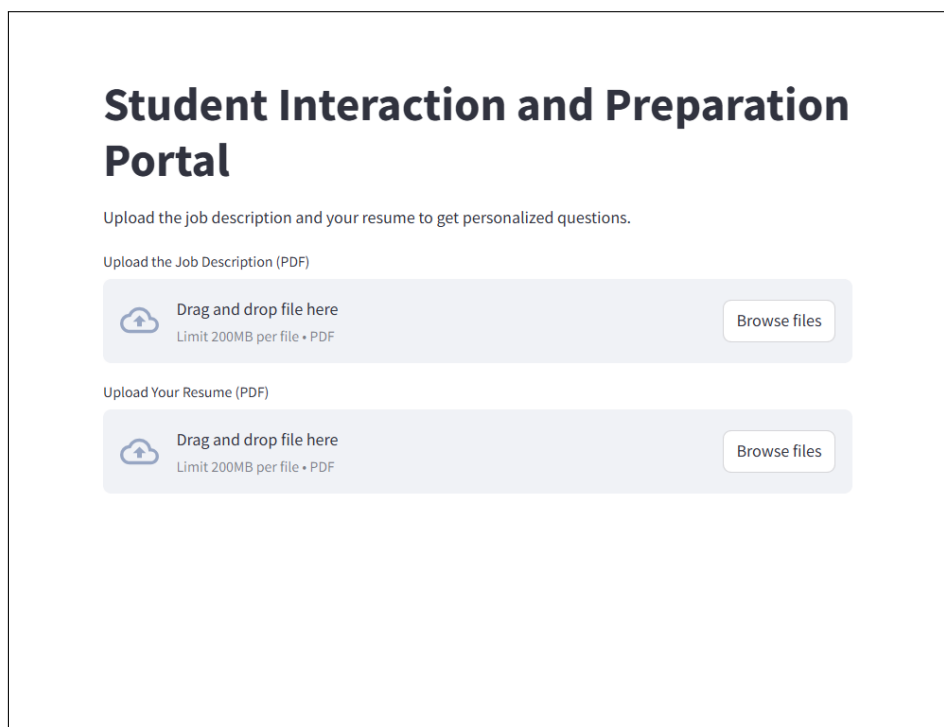
Upload Resume (PDF or Word)

Drag and drop file here
Limit 200MB per file • PDF, DOCX

Browse files

Please upload a file to proceed.

Figure 8.1: Streamlit Interface - Resume Recommender



Student Interaction and Preparation Portal

Upload the job description and your resume to get personalized questions.

Upload the Job Description (PDF)

Drag and drop file here
Limit 200MB per file • PDF

Browse files

Upload Your Resume (PDF)

Drag and drop file here
Limit 200MB per file • PDF

Browse files

Figure 8.2: Streamlit Interface - Student Interaction and Preparation Portal

Q3: Q3. The job description mentions knowledge of Java, Spring Boot, AWS Infra, and Kafka as desirable skills. Although you don't have direct experience with these technologies, how would you approach learning and implementing them in a production environment, and what strategies would you use to ensure your code meets industry-recommended guidelines and best practices?

Your Answer to Q3

Scalability (horizontal & vertical)

Security and maintainability

Rate My Answer for Q3

Rating: I would rate this answer a 6.5 out of 10.

The candidate mentions some general software development principles that are relevant to learning and implementing new technologies, such as modularity, separation of concerns, high cohesion, and low coupling. These principles are important for writing maintainable and scalable code. However, the answer does not directly address how the candidate would approach learning and implementing Java, Spring Boot, AWS Infra, and Kafka in a production environment.

Figure 8.3: Example of High Rating for a Relevant Answer in Preparation Portal

Top 5 job descriptions for the Resume

1. Java Software Engineers develop and maintain software applications using the Java programming language. They write code, debug applications, and collaborate with cross-functional teams to deliver high-quality software solutions. (Index: 1732)
2. Full-Stack Developers are skilled in both frontend and backend development. They can work on all aspects of web application development, from creating user interfaces to designing databases and server infrastructure, providing end-to-end solutions. (Index: 832)
3. Desktop Support Technicians troubleshoot and maintain desktop computer systems, resolving hardware and software problems for users. (Index: 4699)
4. Java Backend Developers specialize in building the server-side components of software applications using Java. They work on database integration, server logic, and performance optimization to ensure efficient and reliable backend functionality. (Index: 3836)
5. IT Support Specialists provide technical assistance to end-users, troubleshooting hardware and software problems, setting up equipment, and maintaining IT infrastructure. (Index: 609)

Figure 8.4: Display of Top 5 Recommended Jobs for User

Questions for You:

Q1: Q1. As a backend application developer, how would you approach optimizing performance for microservices, and what tools or techniques have you used in the past to detect memory leaks and improve performance?

Your Answer to Q1

i dont know

Rate My Answer for Q1

Rating: I would rate this answer a 0.5 out of 10.

The candidate's answer is completely uninformative and lacks any relevant information. They don't even attempt to provide a vague idea of how they would approach optimizing performance for microservices or detect memory leaks. This answer demonstrates a lack of knowledge and preparation for the question, which is a critical aspect of the job.

In a real-world scenario, a backend application developer would need to have a solid understanding of performance optimization techniques, such as caching, load balancing, and parallel processing, as well as tools like New Relic, Datadog, or Prometheus to monitor and analyze performance metrics. They would also need to know how to identify and troubleshoot memory leaks using tools like heap dumps, memory profiling, and garbage collection logs.

This answer is not only uninformative but also shows a lack of effort and preparation, which is a major red flag for any interviewer.

Figure 8.5: User Answering Generated Question in Student Interaction Portal

Analysis Results:

Job Description 1:

Here are the main points explaining why the candidate's resume is a good fit for the Java Software Engineer job description:

- **Relevant programming language skills:** The candidate has experience with Java, which is a key requirement for the job, in addition to other programming languages such as C++, C, and XML.
- **Software development experience:** The candidate has over 25 years of experience in software design and development, with a strong background in developing and maintaining software applications, which aligns with the job's requirements.
- **Collaboration and leadership skills:** The candidate has led software development teams, managed projects, and collaborated with cross-functional teams, demonstrating the ability to work effectively with others, a key aspect of the job.
- **Problem-solving skills:** The candidate has analyzed customer data to determine problem causes and implemented corrective actions, showcasing problem-solving skills, which are essential for a Java Software Engineer role.
- **Familiarity with software development methodologies and tools:** The candidate is familiar with various software development methodologies and tools, including Visual Studio .NET, Adobe Acrobat SDK, and Rational Rose, which are relevant to the job's requirements.

Job Description 2:

Here's why the candidate's resume is a good fit for the Full-Stack Developer job description:

- The candidate has over 25 years of experience in software design and development, which demonstrates their ability to work on all aspects of web application development.

Figure 8.6: Display of LLM-Generated Explanations for Recommended Jobs

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

This project successfully demonstrated the integration of Explainable AI (XAI) into a job recommendation system, enhancing its utility beyond simple matching. The key conclusions drawn are:

1. **Semantic Matching is Effective:** Utilizing sentence embeddings (all-MiniLM-L6-v2) and cosine similarity provides a robust method for matching resumes to job descriptions based on semantic content, overcoming limitations of keyword-based approaches. This leads to potentially more relevant recommendations.
2. **LLMs Enhance Transparency and Utility:** Large Language Models (LLaMA 3 via Groq) proved highly effective in adding value through XAI. They successfully generated concise, relevant explanations for job recommendations, increasing transparency. Furthermore, their ability to generate tailored interview questions and provide feedback on answers creates a powerful tool for candidate preparation.
3. **XAI Integration is Feasible:** The project confirms the feasibility of integrating LLM-based XAI components into a practical application. By carefully crafting prompts and leveraging APIs like Groq, meaningful explanations and interactions can be generated without needing to modify the core recommendation algorithm's internals directly.
4. **Streamlit Facilitates Rapid Prototyping:** Streamlit proved to be an excellent choice for developing an interactive user interface quickly, allowing the focus to remain on the core AI/NLP functionalities while still providing an accessible demonstration platform.
5. **Potential for Improved Job Search Experience:** Combining semantic matching, XAI explanations, and personalized interview preparation offers a significantly enhanced experience for job seekers, providing not just leads but also understanding and targeted practice opportunities.

While the system is a prototype with limitations (static data, API dependency, potential biases), it establishes a strong proof-of-concept for future development of more intelligent, transparent, and user-centric job search platforms.

9.2 Future Work

Several avenues exist for extending and improving this project:

1. **Dynamic Job Data Integration:** Replace the static CSV dataset with integration to live job boards or APIs (e.g., LinkedIn, Indeed) to provide real-time, up-to-date recommendations. Implement efficient updating of job embeddings.
2. **Advanced Embedding Models:** Explore domain-specific embedding models (e.g., fine-tuned on HR/job data) or larger, more powerful general models to potentially improve semantic matching accuracy.

3. **Enhanced Explainability:** Experiment with different XAI techniques or more sophisticated prompting for LLMs to provide deeper, more nuanced explanations. Possibly highlight specific resume sentences/skills that matched job requirements.
4. **User Profiles and Personalization:** Implement user accounts to store resume details, track application history, and personalize recommendations based on user feedback (e.g., thumbs up/down on recommendations).
5. **Skill Gap Analysis:** Extend the system to identify potential skill gaps between the user's resume and desired job descriptions, suggesting areas for development or relevant online courses.
6. **Bias Mitigation:** Incorporate techniques to audit and mitigate potential biases in both the embedding models and the LLM responses to ensure fairer recommendations and evaluations.
7. **Scalability and Performance Optimization:** For larger datasets, utilize optimized vector databases (e.g., FAISS, Milvus) for efficient similarity search instead of in-memory computation. Optimize API calls (e.g., batching if possible).
8. **Robustness and Error Handling:** Improve handling of edge cases, such as poorly formatted resumes, network errors during API calls, or unexpected LLM outputs. Add more comprehensive input validation.
9. **Formal User Evaluation:** Conduct user studies with actual job seekers to gather feedback on the system's usability, the quality of recommendations/explanations, and the effectiveness of the interview preparation module.

9.3 Applications

This XAI-enhanced job recommendation and preparation system has several potential applications:

1. **Online Job Portals:** Enhance existing platforms like LinkedIn, Indeed, or specialized job boards by providing more accurate, explainable recommendations and integrated interview practice tools, improving user engagement and success rates.
2. **University Career Services:** Equip university career centers with a tool to help students and alumni identify suitable job openings, understand why they match, and prepare effectively for interviews based on their academic background and experiences.
3. **Corporate Internal Mobility:** Assist employees within large organizations in finding relevant internal job opportunities, understanding the required skill alignments, and preparing for internal interviews, thus facilitating career growth and retention.
4. **Recruitment Agencies:** Provide recruiters with a tool to quickly assess candidate suitability for multiple roles, understand the rationale behind matches, and potentially pre-screen candidates using tailored questions.

5. **Individual Career Coaching:** Serve as a tool for career coaches to use with clients, helping them refine resumes, target appropriate jobs, understand matching criteria, and practice interview skills with AI feedback.
6. **EdTech Platforms:** Integrate into platforms offering professional development courses, recommending jobs relevant to newly acquired skills and helping users prepare for interviews in their target field.

CHAPTER 10

APPENDIX

10.1 Appendix A: Paper Publication Details

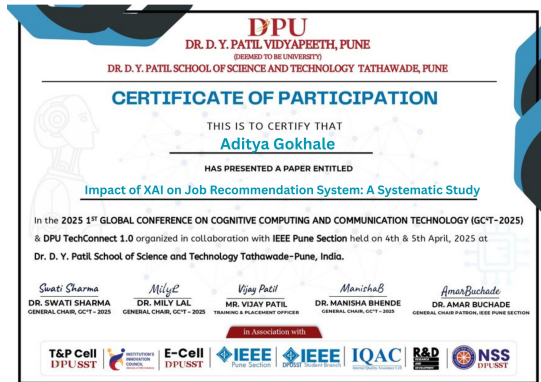


Figure 10.1: Presentation certificate - Aditya Gokhale

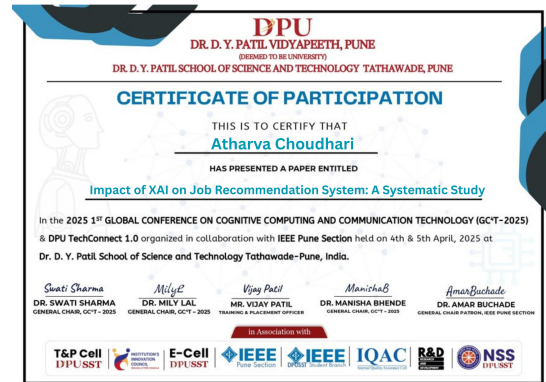


Figure 10.2: Presentation certificate - Atharva Choudhari

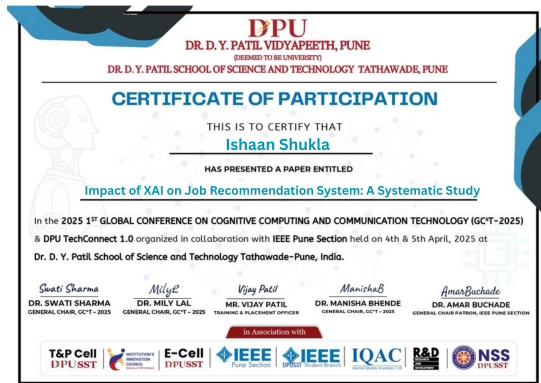


Figure 10.3: Presentation certificate - Ishaan Shukla

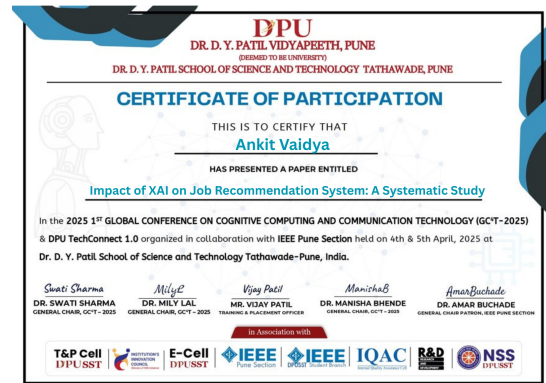


Figure 10.4: Presentation certificate - Ankit Vaidya



Figure 10.5: Presentation certificate - Geetanjali Kale



Figure 10.6: Presentation certificate - Madhuri Patil

Impact of XAI on Job Recommendation System: A Systematic Study

Aditya Gokhale

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
adityangokhale@gmail.com

Atharva Choudhari

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
atharvac1301@gmail.com

Ishaan Shukla

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
ishaanshukla10@gmail.com

Ankit Vaidya

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
ankitvaidya1905@gmail.com

Geetanjali Kale

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
gvkale@pict.edu

Madhuri Patil

Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
mspatil@pict.edu

Abstract—Job recommender systems have become vital tools in the modern recruitment process, assisting both job seekers and employers in navigating a rapidly evolving job market. These systems leverage advanced techniques in artificial intelligence, machine learning, and data mining to provide personalized job recommendations, helping candidates discover relevant opportunities and employers identify suitable talent. This survey provides a comprehensive overview of the state-of-the-art in job recommender systems, including their underlying algorithms, evaluation metrics and the datasets used. The main focus of this work is the study of systems that incorporate explainability and provide novice researchers with a comprehensive view of the current systems.

Index Terms—recommender systems, job recommendation, resume shortlisting, machine learning, knowledge graphs, explainability

I. INTRODUCTION

The employment market has grown more competitive and dynamic in recent years, which has prompted businesses and job seekers to look for innovative ways to match opportunities with talents. In order to expedite the hiring process, job recommender systems and resume shortlisting systems have become ubiquitous and indispensable tools, utilizing the rapid developments in information retrieval, data mining and machine learning. By automatically recommending appropriate job vacancies to people based on their qualifications, preferences, and behavioral data, these systems seek to improve the conventional job search process. They also help organizations identify qualified applicants from large application pools, increasing the hiring process's accuracy and efficiency.

Before the widespread use of the Internet, job recommendation systems like [1] were developed for small-scale usage. Using this system, job seekers could send queries or messages over the telephone line and the service would perform basic text matching algorithms to return a set of potential jobs to the job seeker. Since then the advances in machine learning and information retrieval have led to the development of multiple

systems. These systems can be primarily classified into the following classes - Content-Based Recommender Systems, Collaborative Filtering, Knowledge-Based Systems and Hybrid Systems. [2] and [3] use semantic similarity measures between the posting and resume to choose the most suitable postings for a particular resume. Collaborative filtering is an approach that uses a matrix to model user-item interactions and recommendations are made using a K-Nearest Neighbor (KNN) approach as proposed in [4] and [5]. [6] and [7] use knowledge bases and compute the similarity between resumes and job descriptions in the ontological space to make predictions. Hybrid systems make the use of multiple algorithms or the combination of their outputs to recommend the ideal job posting to a job seeker. Fig.1 shows the classification of job recommendation systems and the various types and algorithms that are used in them.

In the context of job recommendation systems, explainability and interpretability are vital for creating trust between users and the platform. Job seekers need to understand why specific job opportunities are recommended to them, which fosters confidence in the system and encourages engagement. For example, if a system recommends a job based on a user's skills, location, or work history, making these factors clear can help users validate and appreciate the relevance of the suggestions. In addition, employers and recruiters using such platforms must ensure that the algorithms are fair and free from bias, especially with respect to sensitive factors such as gender, age, or ethnicity. Interpretability is also essential for compliance with employment regulations and ethical standards, as unfair or biased recommendations could lead to legal issues. By offering transparency in how job matches are made, job recommender systems can also help users refine their profiles or searches, ultimately improving their experience and increasing the likelihood of better job placements.

There have been many surveys that study job recommendation and resume shortlisting systems like [8], [9] and [10]

which focus on the various systems, the algorithms and datasets used and their results. However, we found that there is a lack of any comprehensive survey of systems that focus on explainability and interpretability. The main motivation behind this paper is to conduct a comprehensive survey on job recommendation systems that explicitly focus on explainability and interpretability to provide researchers and developers with a comprehensive understanding and guide of the field so that future research and development can be aided.

A generalized system architecture for Explainable Recommender Systems is shown in Fig. 3. The phases of the system are- Preprocessing, Training, Evaluation and Explainability on predictions obtained. In the preprocessing phase, different data formats from various sources that are shown in Fig. 2 are aggregated into a single dataset for ease of use and then data is cleaned. Candidate resumes can be in varied formats, so they are processed and the relevant features are extracted. The data is then split into training and testing data. Model(s) are trained on training data and evaluation is done according to metrics used (Euclidean distance, F1-score, Accuracy, etc.). On the predictions generated by system, explainability techniques are used in an attempt to interpret the model's understanding. These results can further be utilized to improve the model by using a feedback loop.

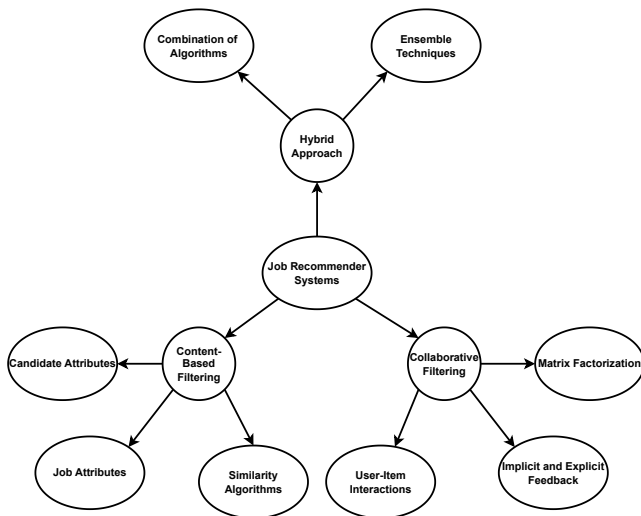


Fig. 1. Types of Recommender Systems

II. LITERATURE SURVEY

A. Non-Explainable Systems

The work done by [11] is one of the first works in the field of resume recommendation systems. They suggest making use of a probabilistic hybrid recommendation engine that ranks the available resumes based on the job profile posted by the recruiter. A latent aspect model is used in order to understand the preferences as a combination of the factors of preference. The recruiter then rates these attributes as per requirement and then ranks the available resumes. However,

this approach makes the use of keywords in order to match and rank the candidates which can fail quickly as the complexity of the resumes and job descriptions increases. Their results established that as sparsity decreased, the Mean Absolute Error also decreased, indicating an increase in prediction accuracy.

A fast-paced approach by [12] was aimed at an automatic resume classification and candidate matching system in an attempt to accelerate the process of selection in high-volume recruiting environments. Addressing the massive volume of applicants, differing resume formats, and requirements for domain-specific knowledge of HR, the proposed methodology uses NLP and machine learning classifiers, particularly a Linear SVM, which effectively classify the resumes and map the candidate with the job description. The system yielded an accuracy of 78.53%, giving a lot of efficiency savings in resume screening and candidate recommendation. Future research will involve the introduction of deep models and the extension for application in support of industry-specific applications, with the aim of moving further in the functioning and improvement of the system for a more versatile and precise action in the real world.

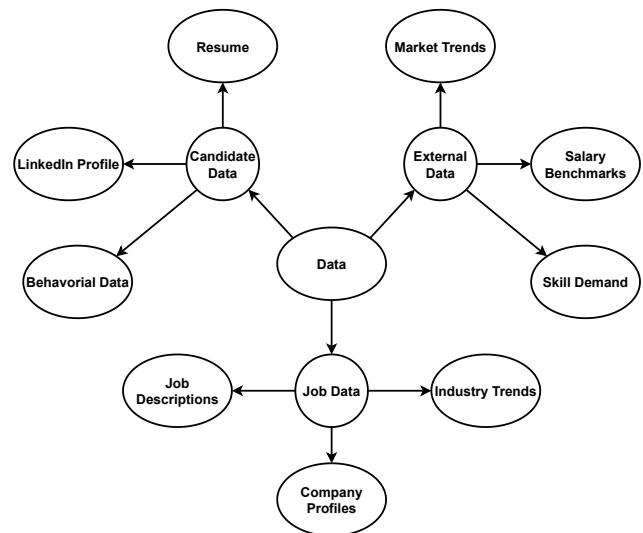


Fig. 2. Overview of data required to build a system

A simple yet strongly effective framework presented by [13] used LLMs for resume screening where the aim was to reach processing speeds 11 times faster than traditional manual reviewing. Fine-tuning LLMs proved capable of classifying sentences into content relevant to resume screening at an F1 score of 87.73%. Broadly, resume screening encompasses information extraction, grading, and summarization, which are among the key functionalities that aid in the holistic assessment of a candidate. The system also shows how agents can be built using LLM which would allow the agent to be able to make decisions in supporting candidate selection, thereby achieving unprecedented productivity advancements that are directly applicable to the recruitment processes of many industries. The accomplishments outlined here indicate promising

TABLE I
SUMMARY OF DIFFERENT METHODOLOGIES: NON-EXPLAINABLE SYSTEMS

Paper	Dataset	Method	Result
(Malinkowski et al.) [11]	100 real life job profiles downloaded from a large German job-portal.	Probabilistic hybrid recommender engine considering preferences of recruiter.	Increase in prediction accuracy with decrease in sparsity.
(Roy et al.) [12]	Downloaded from various online portals and Kaggle.	Using 4 different algorithms, namely Random Forest, Naive Bayes, Logistic Regression and Linear SVM for classification.	Best accuracy (78.53%) on SVM.
(Gan et al.) [13]	Used dataset after manual curation and aggregation from Kaggle.	Using agentic LLM workflow for resume shortlisting.	An F1-score of 87.73%.
(Singh et al.) [14]	Manual annotation of around 110 datasets using GATE (General Architecture for Text Engineering)	Using queries from designed architecture, consisting the components of Batch Processor, Query Processor and Resume Matcher.	Improvement in underlying systems with respect to speed and accuracy.
(Farber et al.) [15]	Downloaded from online job portals.	A hybrid model, consisting of both, content-based and collaborative filtering mechanisms.	Improvement in performance exhibited by hybrid models as compared to individual algorithms.

future potential in the development of LLM-based frameworks for streamlining and improving recruitment workflows.

A support tool, PROSPECT [14], which is a sort of decision support tool designed to be developed for recruiting in order to effectively narrow down and shortlist candidates from large quantities of resumes full of key information, such as skills, experience, and education, in structured formats. Candidate ranking is implemented through advanced information retrieval techniques, aided by the introduction of a novel architecture with 3 key components: Query Processor, Batch Processor and Resume Matcher. This resulted in an improvement in accuracy by 30%, so recruiters will actually be able to focus more on candidates holding specific requirements for the job. It significantly reduces the time needed for screening; therefore, it is one of those streamlined approaches to candidate selection that emphasizes relevance and recruitment optimality.

[15] attempts to improve candidate-job matching of e-recruitment by surmounting the shortcomings of traditional Boolean search methodology, which are mostly incapable of delivering the required degree of candidate-job fit. Personnel selection theory-based probabilistic automated recommendation framework to boost the accuracy of candidate-job requirement matching in recruitment processes was developed by the researchers. Preliminary encouraging results based on synthetic data then provided a basis for improving recruitment outcomes in this approach. The hybrid system performed better than either of its components taken individually. To improve the effectiveness of this framework, more sophistication is needed regarding challenges such as data sparsity and relational attributes in candidate assessments.

B. Explainable Systems

The work of constructing Knowledge Graphs based on the concepts of multi-dimensional knowledge representation was shown in [16] to tackle commonly faced problems in recommender systems like the cold-start problem and the sparsity problem while providing the user explanations for why the job was recommended. Textual templates are created

and the required information is filled in using Named Entity Recognition (NER) techniques. They created a dataset of users and job descriptions from a job-hunting website called “Monster” on which they tested their method. The dataset contained 50 user profiles and around 1660 job postings. To preserve user privacy no personal information was considered in the recommendation algorithm. First, they embed all the documents using paragraph embedding models which give better performance than traditional sentence embedding models. Using these embeddings a similarity matrix is constructed using the cosine-similarity metric. A knowledge graph is constructed where the nodes are the job profiles and the user profiles which contains three types of relationships namely - user-user, user-job, job-job. Using the knowledge graph as a basis for a collaborative filtering approach the recommendations are generated by the system. Dijkstra’s Shortest Path algorithm is used to find the closest nodes to the source nodes which are the generated recommendations. The explanations are generated by using NER techniques and a template is filled with the required extracted data and a similarity score is also provided to the user. The system also lists the relevant terms in the user profile which aided in the recommendation.

The problem of creating interpretable representations to be used by Natural Language Processing models was addressed in [17]. It works on the creation of comprehensive representations of job and user profiles which can be used for effectively interpreting the recommendations suggested by the system. These features are created using feature fusion of explicit and implicit features. They make use of a custom dataset containing 13000 job posts, 580000 candidates and around 1.3 million resumes. They sort the dataset based on the time at which the entries were posted. To learn the feature vector of the job posting they create 2 representations - the explicit representation containing the job requirements and the implicit representation that contains the implicit intention of the recruiter. Similarly, the feature representation of the candidate is learned. Two different neural networks are used to learn

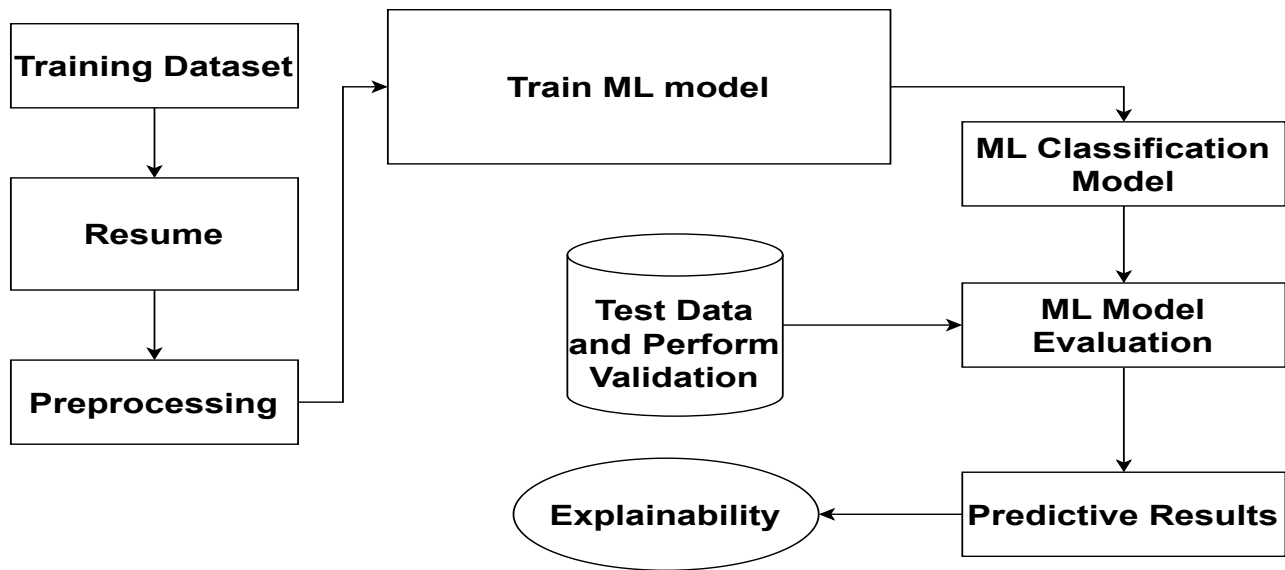


Fig. 3. General System Architecture of an Explainable Recommender System

the features and the features are merged using concatenation. The matching score for the posting and candidate is calculated using an inner product and applying the sigmoid function over it. A Long Short-Term Memory (LSTM) is used in learning the implicit and explicit features. As the feature vectors contain a mix of implicit and explicit features we can easily understand the contributing features for a certain prediction. The use of semantic entities that are extracted from the resumes is also a contributing factor towards greater explainability of this system. This system achieves an accuracy of 92.9 % on the test dataset. This system overcomes the cold starting problem faced by traditional recommender systems.

A novel framework focusing on personalization, DISCO [18] makes use of user's personalized ratings and intentions by modeling their behavior instead of using text-matching frameworks which helps in personalized and more interpretable representations. They use a custom dataset that they collected from an unspecified job recruiting platform. They also make use of a public dataset - Edu-Rec for experimentation. Their system makes use of five distinct components - a base embedding model, a contrastive learning network, a hierarchical skill-aware disentangling module and a diagnosis module. The embedding model embeds the job seekers and job postings in matrices that are trainable. It also makes use of collaborative information and uses a Graph Convolutional Network (GCN) network to extract connectivity. The disentangling module is used to predict the degree of matching between the candidate and the job posting based on the level of interactions. Then a hierarchical module is used to disentangle the representations to increase the interpretability of the system. Then contrastive learning is used in order to enhance the robustness of the representations by exploration of self-supervised signals. The

diagnosis module is then used in order to model the user and posting interactions by introducing cognitive diagnosis theory by studying the interactions between the various skill dimensions. They compare their approach with MF, NGCF, LightGCN and DPGNN beating all the previous methods on the datasets by around 1-2%. The skill representations from their model can effectively show the candidate's mastery of the skills using coarse-grained and fine-grained influences.

[19] consists of training a custom transformer-based architecture. They use a transformer-based architecture and local-global attention to model the interactions of users and job descriptions. This system contains two stages - a ranking stage and a recall stage. They make use of a private dataset provided to them by a company in China and the dataset is not released publicly. In the recall stage, a TextCNN is used to encode the provided job description for item-level representations which are then fed into the transformer to capture the conceptual semantics. The TextCNN is used to reduce a loss of generality as it processes each item separately. The local-global attention is introduced in order to measure the importance of intra-job items in combination with inter-job information. The neighboring job descriptions are calculated using the Euclidean Distance metric and the top-n similar postings are selected to include in the local-global attention mechanism. They make use of the recall stage to predict the skill distribution which is enhanced by a custom loss. To visualize the most crucial skills for any given description they make the use of attention weights with the corresponding user profiles.

A generic approach that uses Part-of-Speech Tagging (POS Tagging), N-grams and TF-IDF are used to recommend jobs [20]. It uses validations and testing of the model on real

data obtained from a top job posting website to show the applicability and efficiency of using topic models as features. A prototype of novel implementation of recommendation algorithm that applies topic modeling technique - Latent Dirichlet Allocation (LDA) [21] used along with feature engineering methods to define features of a job posting and user profile. The engine is natural language processing driven, enriched with n-grams and latent topics as features, it is composed of data pre-processing and context similarity techniques. Several techniques such as Levenshtein distance, fuzzy and semantic matching are performed on the item's data and user's data features. The similarity score is then computed which is further normalized using min-max transformations. A user-job similarity matrix is obtained that determines the closeness of a job to a user. Collaborating filtering models are associated with a cold start problem and there are high chances of observing irrelevant recommendations due to seasonality changes and constantly changing user behaviors. Cosine similarity of the user profile with the job is taken as a metric. Mean recall and F1-measure on the taken data set are 82% and 84 % respectively. The explainability is explained using using TF-IDF vectorizer.

In another interesting work, a framework for career counseling of students using ML and AI techniques was proposed in [22]. ML-based White and Black Box models analyze the educational dataset consisting of academic and employability attributes that are important for the job placements and skilling of the students. The Recall and F-Measure score achieved by the Naive Bayes for performing predictions is 91.2% and 90.7% which is best compared to the score of Logistic Regression, Decision Tree, SVM, KNN, and Ensemble models taken in the study. The system requires input data from students which consists of attributes as follows: a) subject interest of students in senior secondary (Non-Medical, Medical, Arts, Accounts), b) specialization, c) marks obtained in the employability tests, d) work experience, e) status of the place Machine learning is the application of artificial intelligence where the machine will access the data and learn from them. In the proposed work, predictive analytics is performed using the white box and black box machine learning techniques. Apart from this, an in-depth analysis of the White and Black box model techniques is explored for proper explainability and interpreting AI results on a Career counseling-based educational dataset. The accuracy of different machine learning algorithms is compared to perform predictions. The best machine learning algorithm is implemented in the proposed career counseling framework for effective decision-making. SHAP [26] was used to define explainability. Apart from it, there is another XAI framework i.e. known as what-if tool which is an interface for understanding the classification and regression problems related to Black Box Models. A toolkit known as AIX360 i.e. AI explainability 360 developed by IBM and supports model interpretability.

[23] is one of the first works that investigates the role of LLMs in Reasoning Intelligence, especially Prescriptive Intelligence. They address the challenges faced during the matching

of jobs with attributes of talent, like converting unstructured textual data in the form of CVs and JDs (job descriptions) into a structured format. Four approaches are suggested to capture the information that gets lost in the conversion of unstructured text to structured: 1. Content-based deterministic approach which uses traditional techniques 2. LLM-based guided approach 3. LLM-based unguided approach 4. Hybrid approach combining traditional and LLM-based approaches. Experiments were conducted on synthetic data as well as real-world data, which mainly consisted of JDs in the IT domain, to understand the effectiveness and limitations of the approaches used. The Content-based deterministic approach takes unstructured JDs and CV as input and gives recommended JDs with a score as output. In this conversion, talent attributes and job attributes are used, where the former represents the candidate's professional information like preferred role, work experience, skill and proficiency, education, and time zone and the latter represents the corresponding requirements of the job like required role, required experience in the role, required skills and proficiency, preferred education or qualification, preferred time zone. The deterministic algorithm compares various attributes extracted from CV and JD. It uses two comparisons, a Closer Match and an Exact Match. Closer Match awards a score of 1 if the JD attribute matches the CV attribute and the score decreases as absolute deviation increases. Exact Match awards a score of 1 only if the JD attribute is an exact match of the CV attribute, else a score of 0 is given. The Language Model-based approach uses LLMs to extract meaning from text which helps in understanding the correlation between the JD requirements and CV attributes even if the CV attributes do not explicitly match the JD requirements. The LLM-guided algorithm uses a predefined configuration to generate recommendations. The LLM explains its actions in natural language to give reasoning for why a certain recommendation is good or bad. The configuration guides the LLM according to the objective function's criteria. The unstructured JDs and unstructured CV along with the criteria are fed to the LLM in an input prompt and the output from the LLM is the recommended job ID with an explanation. Unlike the guided approach, the LLM unguided algorithm does not use criteria and therefore only the unstructured JDs and unstructured CV are passed to the LLM in an input prompt. The Hybrid approach combines the traditional approach with the LLM-based approach. This is helpful in matching qualitative attributes since traditional techniques can only effectively match quantitative attributes like number of years of experience or proficiency in skills. It uses the traditional deterministic technique to filter a large set of job opportunities to a smaller relevant set and then uses these filtered jobs and focuses on their qualitative attributes using the unguided LLM method. The LLM is prompted to rate the company and the given role in that company on a scale of 1 to 10. The LLM unguided and Hybrid models consistently outperformed other methods, achieving the best overall accuracy rates of 95.66% and 95% for synthetic data, and 89.66% and 87% for real data.

Human-centric approach was explored in [24] where a

TABLE II
SUMMARY OF DIFFERENT METHODOLOGIES: EXPLAINABLE SYSTEMS

Paper	Dataset	Method	Explainability	Result
(C. Upadhyay et. al, 2021) [16]	Custom dataset collected from "Monster" website. 50 user profiles and 1600 job postings.	Knowledge Graphs using multi-dimensional embeddings. Used a collaborative filtering approach using Dijkstra's algorithm.	Using NER to extract relevant attributes and a similarity score is provided.	Average of 26% on BELU and Rouge-L scores.
(J. Jiang et. al, 2020) [17]	Custom Dataset. 13000 job postings, 580,000 candidates and 1.3 million resumes.	Create features using feature-fusion. LSTM used to provide recommendations.	Semantic and explicit features which the LSTM focuses on.	Accuracy score of 92.9% on test dataset.
DISCO (X. Yu et. al, 2024) [18]	Open source dataset, EduRec, containing 5,230 ratings given by 476 users over 70 items.	Five distinct components - a base embedding model, a contrastive learning network, a hierarchical skill aware disentangling module, a diagnosis module and a Graph Convolutional Network.	Using the diagnostic module defined.	Beat previous methods on the datasets by around 1-2 %.
(Z. Guan et. al, 2024) [19]	Private dataset by a company in China which has not been released publicly yet.	TextCNN for encodings and transformer-based architecture for conceptual semantics.	Top 'n' matching jobs are computed using Euclidean distance.	Beat previous best systems with a 4-5 margin on recall and 2-3 margin on NDCG.
(S. Bansal et. al, 2017) [20]	Two data sets - user details and jobs data. Complete corpus of 32,163 job data and 11244 unique users are obtained.	Uses Latent Dirichlet Allocation with feature engineering methods.	Similarity score is shown using techniques like Levenshtein distance, fuzzy and semantic matching.	Mean recall and F1-measure of 82% and 84 % respectively
(P. Guleria et. al, 2022) [22]	Private Educational Dataset	Several models like Naive Bayes, Logistic Regression, Decision Tree, SVM, KNN and ensembling techniques are used.	Mase use of SHAP library for explainability and also used the toolkit AIX360 by IBM.	Accuracy scores of 91.16% when using Naive Bayes algorithm.
(P. Ghosh et. al, 2023) [23]	Used synthetic as well as real-world custom dataset (mostly consisting of JDs from IT domain).	Four approaches as: Content-based deterministic, LLM-based guided, LLM-based unguided and Hybrid approach.	LLM explanations in natural language.	LLM unguided and Hybrid models (best approach) achieved accuracy rates of 95.66% and 95% for synthetic data, and 89.66% and 87% for real data.
(T. Tran, 2023) [24]	Kaggle's Career Builder 2012 dataset,	Use of White Box models (Logistic Regression, Decision Tree, Gaussian Naive Bayes) and Black Box models (XGBoost, AdaBoost, LDA, QDA).	Libraries of LIME and Kernel SHAP for explainability for post-hoc explainability and Explainable Boosting Machines (EBM) and Differentially Private Explainable Boosting Machines (DPEBM) for model-specific explainability.	All models gave a high accuracy score and a high recall (around 80%).
(J. Choi et. al, 2020) [25]	52,190 job advertisements from a job posting website	LSTM to train the classifier including text preprocessing	Libraries of LIME	Accuracy score of about 80% using LSTM.

three-step process to map methodology to four research questions which include building a job recommender system, using XAI techniques and evaluation metrics for explainability along with human-centric explanation was followed. The construction of the job recommender system includes feature engineering, filtering potential job applications for each candidate, ranking job applications and producing the final results. TF-IDF and LDA were used in feature engineering. After the transformation from feature engineering, White Box ranking algorithms (Logistic Regression, Decision

Trees, Gaussian Naive Bayes) and Black Box ranking algorithms (XGBoost, AdaBoost, Linear Discriminant Analysis, Quadratic Discriminant Analysis) were used. The next step is filtering potential applications which involves ranking data and it is carried out with two approaches: random sampling with control of positive labels and unsupervised KNN. A maximum of 100 applications are filtered for each user, to simplify the ranking. Given the binary nature of the outcome (apply or not apply), binary classification models are employed. LIME [27] and Kernel SHAP were

used for post-hoc explainability. Explainable Boosting Machines (EBM) [28] and Differentially Private Explainable Boosting Machines (DPEBM) [29] were employed for model-specific explanations. EBM is tailored for tree-based models, while DPEBM is an enhanced version of EBM with added privacy protections. Evaluation for explanation of job recommendations was done by calculating the model fidelity rate for each candidate and feature importance fidelity rate.

There are also approaches that make use of LSTM for classification was used by for the domain of IT jobs [25]. The Python library, LIME was used for explainability of outputs. Text classification was performed to classify job advertisements and LIME was applied to find the terms that were considered important factors for each category of job roles. An accuracy of around 80% was achieved using LSTM.

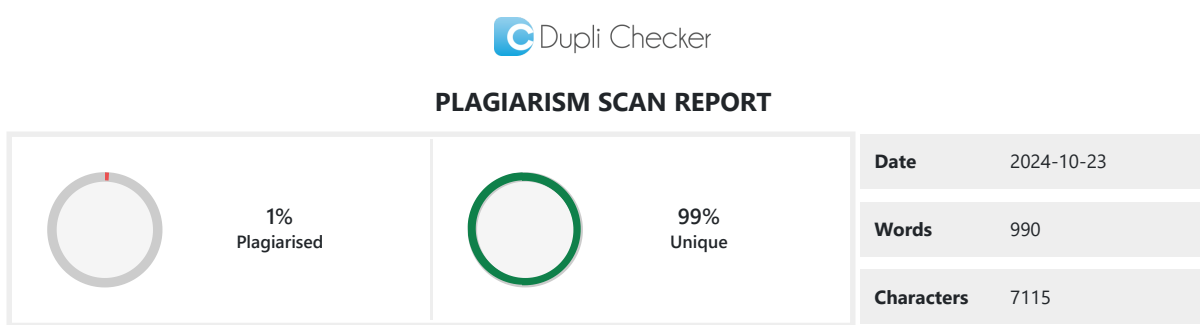
III. CONCLUSION

Job recommender systems, by harnessing the power of artificial intelligence, machine learning, and data mining, offer personalized job recommendations, improving the discovery of relevant opportunities for candidates and the identification of suitable talent for employers. We found that Knowledge Graphs were the most used in systems for explainability and fetched good results. Other models (CNNs, transformer-based architectures, LLMs and other custom traditional and modern systems) also fared comparable results. Despite significant advancements, challenges such as addressing user preferences, mitigating bias, and ensuring transparency remain critical. As the field progresses, future efforts will likely focus on enhancing the accuracy, fairness, and overall user experience of these systems, ensuring they continue to evolve alongside the changing needs of the workforce.

REFERENCES

- [1] J. Vega, "Semantic matching between job offers and job search requests," in *COLING 1990 Volume 1: Papers presented to the 13th International Conference on Computational Linguistics*, 1990.
- [2] T. V. Yadalam, V. M. Gowda, V. S. Kumar, D. Girish, and N. M., "Career recommendation systems using content based filtering," in *2020 5th International Conference on Communication and Electronics Systems (ICCES)*, pp. 660–665, 2020.
- [3] N. D. Almalis, G. A. Tsihrintzis, N. Karagiannis, and A. D. Strati, "Fodra-a new content-based job recommendation algorithm for job seeking and recruiting," in *2015 6th International Conference on Information, Intelligence, Systems and Applications (IISA)*, pp. 1–7, 2015.
- [4] Y. Lee, Y.-C. Lee, J. Hong, and S.-W. Kim, "Exploiting job transition patterns for effective job recommendation," in *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, p. 2414–2419, IEEE Press, 2017.
- [5] S. Ahmed, M. Hasan, M. N. Hoq, and M. A. Adnan, "User interaction analysis to recommend suitable jobs in career-oriented social networking sites," in *2016 International Conference on Data and Software Engineering (ICoDSE)*, pp. 1–6, 2016.
- [6] A. B. Kmail, M. Maree, M. Belkhatir, and S. M. Alhashmi, "An automatic online recruitment system based on exploiting multiple semantic resources and concept-relatedness measures," in *2015 IEEE 27th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 620–627, 2015.
- [7] A. B. Kmail, M. Maree, and M. Belkhatir, "Matchingsem: Online recruitment system based on multiple semantic resources," in *2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*, pp. 2654–2659, 2015.
- [8] C. de Ruijt and S. Bhulai, "Job recommender systems: A review," 2021.
- [9] Y. Mashayekhi, N. Li, B. Kang, J. Lijffijt, and T. De Bie, "A challenge-based survey of e-recruitment recommendation systems," *ACM Comput. Surv.*, vol. 56, June 2024.
- [10] R. Thali, S. Mayekar, S. More, S. Barhate, and S. Selvan, "Survey on job recommendation systems using machine learning," in *2023 International Conference on Innovative Data Communication Technologies and Application (ICIDCA)*, pp. 453–457, 2023.
- [11] J. Malinowski, T. Keim, O. Wendt, and T. Weitzel, "Matching people and jobs: A bilateral recommendation approach," in *Proceedings of the 39th Annual Hawaii International Conference on System Sciences (HICSS'06)*, vol. 6, pp. 137c–137c, 2006.
- [12] P. K. Roy, S. S. Chowdhary, and R. Bhatia, "A machine learning approach for automation of resume recommendation system," *Procedia Computer Science*, vol. 167, pp. 2318–2327, 2020. International Conference on Computational Intelligence and Data Science.
- [13] C. Gan, Q. Zhang, and T. Mori, "Application of llm agents in recruitment: A novel framework for resume screening," *ArXiv*, vol. abs/2401.08315, 2024.
- [14] A. Singh, C. Rose, K. Visweswariah, V. Chenthamarakshan, and N. Kambhatla, "Prospect: a system for screening candidates for recruitment," in *Proceedings of the 19th ACM International Conference on Information and Knowledge Management, CIKM '10*, (New York, NY, USA), p. 659–668, Association for Computing Machinery, 2010.
- [15] F. Färber, T. Weitzel, and T. Keim, "An automated recommendation approach to selection in personnel recruitment," in *Americas Conference on Information Systems*, 2003.
- [16] C. Upadhyay, H. Abu-Rasheed, C. Weber, and M. Fathi, "Explainable job-posting recommendations using knowledge graphs and named entity recognition," in *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 3291–3296, 2021.
- [17] J. Jiang, S. Ye, W. Wang, J. Xu, and X. Luo, "Learning effective representations for person-job fit by feature fusion," in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management, CIKM '20*, (New York, NY, USA), p. 2549–2556, Association for Computing Machinery, 2020.
- [18] X. Yu, C. Qin, Q. Zhang, C. Zhu, H. Ma, X. Zhang, and H. Zhu, "Disco: A hierarchical disentangled cognitive diagnosis framework for interpretable job recommendation," 2024.
- [19] Z. Guan, J.-Q. Yang, Y. Yang, H. Zhu, W. Li, and H. Xiong, "Jobformer: Skill-aware job recommendation with semantic-enhanced transformer," 2024.
- [20] S. Bansal, A. Srivastava, and A. Arora, "Topic modeling driven content based jobs recommendation engine for recruitment industry," *Procedia Computer Science*, vol. 122, pp. 865–872, 2017. 5th International Conference on Information Technology and Quantitative Management, ITQM 2017.
- [21] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, p. 993–1022, Mar. 2003.
- [22] P. Guleria and M. Sood, "Explainable ai and machine learning: performance evaluation and explainability of classifiers on educational data mining inspired career counseling," *Education and Information Technologies*, vol. 28, p. 1081–1116, July 2022.
- [23] P. Ghosh and V. Sadaphal, "Jobrecogpt – explainable job recommendations using llms," 2023.
- [24] T. Tran, "Explainable artificial intelligence in job recommendation systems," August 2023.
- [25] I. H. Choi, Y. S. Kim, and C. K. Lee, "A study of the classification of it jobs using lstm and lime," in *The 9th International Conference on Smart Media and Applications*, pp. 248–252, 2020.
- [26] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," 2017.
- [27] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," 2016.
- [28] Y. Lou, R. Caruana, J. Gehrke, and G. Hooker, "Accurate intelligible models with pairwise interactions," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '13*, (New York, NY, USA), p. 623–631, Association for Computing Machinery, 2013.
- [29] H. Nori, R. Caruana, Z. Bu, J. H. Shen, and J. Kulkarni, "Accuracy, interpretability, and differential privacy via explainable boosting," 2021.

10.2 Appendix B: Plagiarism Report



Content Checked For Plagiarism

In recent years, ML has transformed a wide variety of industries by enabling complex decision-making processes that rely on patterns learnt from data. The impact has only increased ever since the advent of LLMs. However, with the increase in sophistication of models in line with recent advancements in the field, they become opaquer, leading to concerns about lack of transparency in how they arrive at their decisions. This is especially true for LLMs which are still considered to be “black box” models. They are called so because the models are created directly from data and no engineers or data scientists who create the algorithm can explain how the AI algorithm arrived at a specific result.

Explainability refers to the ability to describe the internal workings of an ML model in human-understandable terms. This is sometimes also aided with pictures, graphs and charts to help visualize the model’s understanding. Interpretability focuses on the degree to which humans can understand and predict the model’s behavior. The two terms are related closely and even used interchangeably, but they highlight different aspects of model transparency. The major challenges to transparency include complexity of internal mechanics, multi-layer abstractions and lack of feature transparency.

Resume shortlisting is one of the critical steps in recruitment process where traditionally recruiters have to evaluate a large pool of candidates to assess their skills and relevancy to role they are applying for. This process can be automated with the use of ML algorithms like logistic regression, decision trees, random forest and neural networks. More advanced approaches involve usage of models like BERT, GPT-4 and other transformer-based architectures.

However, when automated screening is involved, issues can arise related to the decision-making steps followed by the ML model. In the recruitment process, PICT, Department of Computer Engineering 2024-25

2

transparency plays a vital role so that both employers and candidates understand why and how the decision was made. With explainability, the answers to key questions like: ‘Why was the particular applicant accepted or rejected?’ and ‘What factors strongly affected the model’s decision?’ can be answered. The more “black box” leaning a model is, the harder is trusting the model’s decision and hence transparency is undermined. A lack of explainability can make it difficult for both recruiters and candidates to trust the model’s recommendations.

The approaches to explainable AI are: feature importance, attribution and visualization. The choice between them is made depending on the context and application domain. Implementation of xAI in Python is improved by libraries such as

LIME, SHAP, ELI5 and Captum. LIME (based on predictions of ML model) is useful to provide instance-level explanations but focuses on local behavior. SHAP (based on Shapley values from game theory) is capable of explaining both local and global behavior. Captum is specifically useful for PyTorch models and is best suited for neural network based architectures. ELI5 provides local and global explanations supporting a range of models like linear models, decision trees, and ensemble methods like random forest.

1.1 Motivation

For both lightweight (Logistic Regression, Random Forest, Decision Tree) and advanced algorithms (Neural Network, BERT-based Models, LLMs), the models can develop an inaccurate representation during training phase. This leads to challenges like lack of context understanding, bias amplification, overemphasis on keywords, difficulty with non-traditional careers, limited understanding of soft skills and inability to explain decisions made. Introducing explainability will help enable transparency and address the challenges and also help improve the model further using feedback. The key motivation is to overcome the existing limitations like transparency in decision-making, bias detection and mitigation, continuous improvement, regulatory compliance and improved candidate feedback. This will have 2 main benefits, namely, the candidate will understand the expected

PICT, Department of Computer Engineering 2024-25
3

qualifications and where the bar is set to get selected for a particular job role at a company and it will make the screening process easier for companies and help them focus on assessing suitable candidates only, thus saving time and resources.

1.2 Problem Statement

ML models are widely used for resume shortlisting as a result of the growing use of automation systems in hiring. Even though the models are effective and scale efficiently, their frequent functioning as “black box” models makes it challenging for both candidates and hiring managers to comprehend the reason behind selection choices. This raises critical issues like unintended biases, decline in systematic trust, and the possibility of breaking legal frameworks that require responsibility and fairness in hiring practices. Explainability techniques that can offer clear and interpretable insights into the decision-making process, which is desperately needed in context of resume shortlisting. The main motive is to try to understand the working inside the “black box” models and hence improve the system.

Multiple research endeavors have concentrated on using a multitude of methods like rule-based systems, machine learning methods to classify resumes have been undertaken which address the complex challenges in this field. This survey aims to provide a concise review of all the earlier efforts which were undertaken. A thing to note is that our work is one of the first ones to make the use of state-of-the-art model explainability and interpretability methods to explain the complex decisions these models make, understanding the earlier works can help provide the reader a perspective of the past developments in the field.

The work done by Malinkowski et al. [1] is one of the first works in the field of resume recommendation systems. They suggest making the use of a probabilistic hybrid recommendation engine which ranks the available resumes based on the job profile posted by the recruiter. A latent aspect model is used in order to understand the preferences as a combination of the factors of preference. The recruiter then rates these attributes as per requirement and then ranks the available resumes. However, this approach makes the use of keywords in order to match and rank the candidates which can fail quickly as the complexity of the resumes and job descriptions increases.

Matched Source

Similarity 7%

Title: [Continuous integration - definition & overview](#)

This process can be automated with the use of a continuous integration server that monitors the code repository for changes and automatically builds and ...

<https://www.sumologic.com/glossary/continuous-integration>

Check By:  Dupli Checker

CHAPTER 11

REFERENCES

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998-6008). (Foundational paper for Transformers)
- [2] Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 3982-3992). (Introduced Sentence Transformers)
- [3] Aggarwal, C. C. (Ed.). (2016). *Recommender systems: the textbook*. Springer. (General overview of recommendation system techniques, including content-based)
- [4] J. Malinowski, T. Keim, O. Wendt and T. Weitzel, "Matching People and Jobs: A Bilateral Recommendation Approach," *Proceedings of the 39th Annual Hawaii International Conference on System Sciences (HICSS'06)*, Kauai, HI, USA, 2006, pp. 137c-137c, doi: 10.1109/HICSS.2006.266.
- [5] Chirag Daryani, Gurneet Singh Chhabra, Harsh Patel, Indrajeet Kaur Chhabra, Ruchi Patel(2020).An Automated Resume Screening System Using Natural Language Processing And Similarity. *Topics In Intelligent Computing And Industry Design*, 2(2): 99-103.
- [6] Färber, Frank, Tim Weitzel and Tobias Keim. "An Automated Recommendation Approach to Selection in Personnel Recruitment." *Americas Conference on Information Systems* (2003).
- [7] Amit Singh, Catherine Rose, Karthik Visweswariah, Vijil Chenthamarakshan, and Nandakishore Kambhatla. 2010. PROSPECT: a system for screening candidates for recruitment. In *Proceedings of the 19th ACM international conference on Information and knowledge management (CIKM '10)*. Association for Computing Machinery, New York, NY, USA, 659–668.
- [8] S. Bharadwaj, R. Varun, P. S. Aditya, M. Nikhil and G. C. Babu, "Resume Screening using NLP and LSTM," 2022 International Conference on Inventive Computation Technologies (ICICT), Nepal, 2022, pp. 238-241, doi: 10.1109/ICICT54344.2022.9850889.
- [9] S. Mhatre, B. Dakhare, V. Ankolekar, N. Chogale, R. Navghane and P. Gotarne, "Resume Screening and Ranking using Convolutional Neural Network," 2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS), Coimbatore, India, 2023, pp. 412-419, doi: 10.1109/ICSCSS57650.2023.10169716.
- [10] P. Ghosh and V. Sadaphal, 'JobRecoGPT – Explainable job recommendations using LLMs', arXiv [cs.AI]. 2023.
- [11] Du, Y., Luo, D., Yan, R., Wang, X., Liu, H., Zhu, H., Song, Y. and Zhang, J. 2024. Enhancing Job Recommendation through LLM-Based Generative Adversarial Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*. 38, 8 (Mar. 2024), 8363-8371.

- [12] Gan, Chengguang, Qinghao Zhang and Tatsunori Mori. “Application of LLM Agents in Recruitment: A Novel Framework for Resume Screening.” ArXiv abs/2401.08315 (2024): n. Pag.
- [13] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM computing surveys (CSUR)*, 51(5), 1-42. (Survey on XAI methods)
- [14] Streamlit Inc. (2024). *Streamlit Documentation*. Retrieved from <https://docs.streamlit.io/> (Tool Documentation)
- [15] Groq Inc. (2024). *Groq API Documentation*. Retrieved from <https://console.groq.com/docs/> (Tool Documentation)
- [16] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*. (Seminal paper on capabilities of large language models like GPT-3)
- [17] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., ... Wen, J. R. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*. (Overview of LLMs)
- [18] Poria, S., Hazarika, D., Majumder, N., Naik, G. (2017). Context-dependent sentiment analysis in user-generated videos. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 873-883). (Example application of NLP in user content)
- [19] Lops, P., de Gemmis, M., Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In *Recommender systems handbook* (pp. 73-105). Springer, Boston, MA. (Detailed look at content-based methods)
- [20] Manning, C. D., Raghavan, P., Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (Foundational text covering concepts like TF-IDF and vector space models)

LIST OF ABBREVIATIONS

Abbreviation	Illustration
API	Application Programming Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DOCX	Office Open XML Document (Word)
ERD	Entity Relationship Diagram
GPU	Graphics Processing Unit
GPT	Generative Pre-trained Transformer
GUI	Graphical User Interface
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JD	Job Description
LLM	Large Language Model
NLP	Natural Language Processing
OS	Operating System
PDF	Portable Document Format
PII	Personal Identifiable Information
RAM	Random Access Memory
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
UML	Unified Modeling Language
XAI	Explainable Artificial Intelligence

graphicx

LIST OF FIGURES

Figure	Illustration	Page No.
Frontispiece	PICT Logo	i
Frontispiece	PICT Logo on Certificate	ii
4.1	DFD Level 0: System Context Diagram	23
4.2	DFD Level 1: Job Recommendation Process	24
4.3	Use Case Diagram	26
4.4	Activity Diagram: Job Recommendation Flow	27
4.5	Sequence Diagram: Generating Recommendation Explanation	28
8.1	Streamlit Interface - Resume Recommender	49
8.2	Streamlit Interface - Student Interaction and Preparation Portal	49
8.3	Example of High Rating for a Relevant Answer in Preparation Portal	50
8.4	Display of Top 5 Recommended Jobs for User	50
8.5	User Answering Generated Question in Student Interaction Portal	51
8.6	Display of LLM-Generated Explanations for Recommended Jobs	51

LIST OF TABLES

Table	Illustration	Page No.
3.1	Software Requirements	17
3.2	Hardware Requirements	18
5.1	RISK ANALYSIS TABLE	34
App.1	LIST OF ABBREVIATIONS	72
App.2	LIST OF FIGURES	73
App.3	LIST OF TABLES	74